GitHub Copilot for Programmers

Eric W. Greene
eric@training4programmers.com

Table of Contents

Welcome!

Module 1: Copilot Extension

- GitHub Copilot Editor Extensions
- Demo: C++ Console App in VS Code
- Demo: Java Console App in VS Code
- Demo: Java Spring REST API with IntelliJ IDEA
- Demo: Python Console App in VS Code
- Demo: C# ASP.NET MVC Web API with Visual Studio
- Demo: React TypeScript Web App with Visual Studio Code

Module 2: Copilot Suggestions

- Practical Copilot Techniques for Visual Studio Code
- Useful Copilot Chat Techniques for Visual Studio Code
- Explore Non-Deterministic Code Suggestions
- Demo: C++ Code Suggestions
- Demo: Java Code Suggestions

Module 3: VS Code Copilot Scopes

- VS Code Copilot Prompts
- Terminal Copilot Chat
- Terminal Copilot & Containerization

Module 4: Prompts and Suggestions

- C++ X-Shot Prompting
- Java X-Shot Prompting
- Python X-Shot Prompting

Module 5: Copilot and Workspaces

- Workspace New Command for a C++ Project
- Workspace Scoped Prompts for a C++ Project
- Workspace New Command for a Java Project
- Workspace New Command for a Python Project

Module 6: Discuss Code with Copilot

- C++: Advice on Getters/Setters
- Java: Equivalent of C# Like Properties
- Java: Coding Conventions
- Python: Improve Loop Efficiency
- C#: Pattern Matching
- C#: Null References

Module 7: Fix Code with Copilot
- C++: Convert Int to String
- C++: Detect a Memory Leak
- C++: Detect a Dangling Pointer
- C++: Detect a Buffer Overflow
- C++: Detect Undefined Behaviors
- C++: Detect Resource Leaks
- C++: Detect Improper Use of Pointers and References
- C++: Detect Memory Allocation Failures
- Java: Incorrect String Concatenation
- Java: Equality Confusion
- Python: Convert Int to String
- Python: Dictionary Key Error
- C#: Index Out of Range
- C#: Async without Await
- C#: Blocking Call

Module 8: Workshops
- Build a Calculator with C++
- C++: Write a Unit Test
- Build a Calculator with Java
- Build a Calculator with C#

Conclusion

Welcome!

Welcome to the GitHub Copilot for Programmers training! The following content is the courseware for this course. The course delivery is 100% live coding. Students are encouraged to code with the instructor if they have Copilot ready in Visual Studio Code and optionally in other editors.

Below are the course modules and resource links used in the course. We will not use every demo and resource in your training session, but you are encouraged to explore all the resources following class to continue learning.

Module 1: Copilot Extension

GitHub Copilot is a web-based service that requires a GitHub account and active license via subscription. Your GitHub administrator assigns licenses. In addition to the subscription, you must install the GitHub Copilot extension(s) in your IDE.

The following links help start with GitHub Copilot in the officially supported editors. Not all editors support all features of GitHub Copilot. The links include instructions on how to install the extension as well as how to use it.

Some editors require two extensions: one for the Copilot suggestions and one for the chat feature. The chat feature is a way to interact with Copilot to get help, ask questions, and get code suggestions. The chat feature is not available in all editors. JetBrains IDEs support both code suggestions and chat using one plugin.

GitHub Copilot Editor Extensions

- Getting Started with Copilot Editor Extensions
  (https://docs.github.com/en/copilot/using-github-copilot/getting-started-with-github-copilot)
- Getting Started with Copilot Chat Editor Extensions
  (https://docs.github.com/en/copilot/github-copilot-chat/using-github-copilot-chat-in-your-ide)

Demo: C++ Console App in VS Code

This is a demo to show how to use GitHub Copilot to help you write a new C++ program. These are suggested prompts to use with GitHub Copilot. Feel free modify them or add additional prompts to get the desired results.

1. Create a "Hello, World!" Program

```
TEXT
Hello, I am new to C++ programming, but I do have computer programming experience. I would like to create a simple "Hello, World!" program in C++. Can you help me?
```

2. Learn about Build Tools for C++ Project

```
TEXT
Does C++ support any build configuration tools? If so, what are they and how do I use them? Which one do you recommend?
```

3. Configure the C++ Project to Use CMake

```
TEXT
Please explain how to configure a C++ project to use CMake. I would like an example of the CMakelists.txt file and how to build and run the project with CMake.
```

4. Add Unit Testing to the C++ Project.

```
TEXT
A friend of mine told me that I should add unit testing to my C++ project. Can you help me with that? Which C++ unit testing framework would you recommend?
```

5. Add Google Test to the C++ Project.

```
TEXT
Show me how to incorporate Google Test into the CMake build of my C++ project. Also, show me how to write a simple test case using Google Test.
```

6. Setup Source Control for the C++ Project.

```
TEXT
I need to work on my project on multiple computers and track changes to my code. Is there a programming tool that I could use for this? If so, which one do you recommend?
```

7. Setup Git for the C++ Project.

```
TEXT
Explain to me how to setup Git for my C++ project. I would like to know how to create a new repository, add files to the repository, and commit changes to the repository. Also, how do I exclude all of the files generate by CMake?
```

Demo: Java Console App in VS Code

This is a demo to show how to use GitHub Copilot to help you write a new Java program. These are suggested prompts to use with GitHub Copilot. Feel free modify them or add additional prompts to get the desired results.

1. Create a Java Hello World Program

```
TEXT
```

```
Hello, I am new to Java. I am working on a new Java Hello World program. How do I create a simple Java program file, and then compile and run the program?
```

2. Learn about Build Tools for Java Project

```TEXT
Does Java support any build configuration tools such as CMake for C++ or Conda for Python?
```

3. Configure the Project to Use Maven

```TEXT
How do I configure my project to use Maven?
```

4. How to Run a Project with Maven

```TEXT
How do I run the project with Maven?
```

Demo: Java Spring REST API with IntelliJ IDEA

This is a demo to show how to use GitHub Copilot to help you write a new Java Spring Boot REST API application. These are suggested prompts to use with GitHub Copilot. Feel free modify them or add additional prompts to get the desired results.

1. Install the GitHub Copilot Plugin for IntelliJ IDEA. Optionally, disable the JetBrains AI plugin to avoid conflicts with GitHub Copilot.
2. Open IntelliJ IDEA, and create a new project with the following options:
   - Generator: Spring Initializr
   - Name: DemoWebApi
   - Group: com.training4programmers
   - Language: Java
   - Type: Maven
   - Java Version 21 (or latest)

Click `Next`. Under `Dependencies`, under the `Web` category, select `Spring Web`. Click `Create`. A new

Spring Web Project will be created.

3. On the right-side activity bar, open the GitHub Copilot chat panel. Also, ensure the Java file `DemoWebApiApplication.java` located in the `src/main/java/com.training4programmers.demowebapi` directory is open. GitHub Copilot will only use files opened in the editor when formulating prompts to the GitHub Copilot service to generate suggestions.
4. Let’s add a new "Hello, World!" REST API endpoint. In the Copilot chat window, type the following prompt:

Copilot Chat Prompt

```text
I need to implement a Hello World REST API endpoint accessible on the path "/hello". How do I add this to my project?
```

Copilot Chat Response

To implement a Hello World REST API endpoint in your Spring Boot application, you need to create a new controller class. This class will handle the HTTP requests coming to the "/hello" path.

Here's a step-by-step plan:

1. Create a new class named `HelloController` in the same package as your main application class.
2. Annotate this class with `@RestController` to indicate that it's a controller class.
3. Define a method named `helloWorld` that will handle GET requests to the "/hello" path. Annotate this method with `@GetMapping("/hello")`.
4. This method should return a `String` that says "Hello, World!".

Here's the code for the `HelloController` class:

HelloController.java

```java
package com.training4programmers.demowebapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String helloWorld() {
        return "Hello, World!";
    }
}
```

After adding this class to your project, you can start your application and access the endpoint at `http://localhost:8080/hello`. It will return the string "Hello, World!".

5. Highlight the following code:

HelloController.java

```java
@GetMapping("/hello")
public String helloWorld() {
    return "Hello, World!";
}
```

Enter the following prompt in the Copilot chat window:

Copilot Chat Prompt

```text
/test create a new unit test
```

Copilot Chat will reply with something like this:

Copilot Chat Response

```text
```

To create a unit test for your HelloController class, you can use the JUnit and Mockito frameworks. Here's a step-by-step plan:

1. Create a new Java class named HelloControllerTest in the com.training4programmers.demowebapi package under your test directory (usually src/test/java).
2. Annotate this class with @RunWith(SpringRunner.class) to indicate that it should use Spring's testing support.
3. Annotate this class with @WebMvcTest(HelloController.class) to indicate that it's a test for HelloController.
4. Autowire a MockMvc instance. This class is the main entry point for server-side Spring MVC test support.
5. Create a new test method named testHelloWorld that doesn't return anything (void).
6. In this method, use the mockMvc.perform() method to send a GET request to the "/hello" path.
7. Assert that the response status is OK and the content is "Hello, World!".

Here is the code for the HelloControllerTest class:

Copilot Proposed Change: HelloControllerTest.java

```java
package com.training4programmers.demowebapi;

import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.junit4.SpringRunner;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@RunWith(SpringRunner.class)
@WebMvcTest(HelloController.class)
public class HelloControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testHelloWorld() throws Exception {
        mockMvc.perform(get("/hello"))
                .andExpect(status().isOk())
                .andExpect(content().string("Hello, World!"));
    }

}
```

After copying the suggested code into the new `HelloControllerTest.java` file, use the IntelliJ in-editor tooling (red light bulb) to install JUnit 4. Once installed, click the green arrow next to the unit test method to run the test. It should pass and verify that the `HelloController` class is working as expected.

6. Type the following comment following the `helloWorld` method.

HelloController.java

```java
@GetMapping("/hello")
public String helloWorld() {
    return "Hello, World!";
}

// GET /hello/{message}
```

After typing the comment, type `<ENTER>`, a suggestion should appear. Accept the suggestions to create the method. The new method should look similar to this.

HelloController.java

```java
// GET /hello/{message}
@GetMapping("/hello/{message}")
public String helloMessage(String message) {
    return "Hello, " + message + "!";
}
```

7. Create a new controller to manage a list of color objects. In the Copilot chat window, type the following prompt:

Copilot Chat Prompt

```text
I need a service to manage a list of color objects. Each color will have an id, a name, and a hexcode. Please generate a Color class, a service class to manage a list of colors, including adding, replacing, and deleting a color. In the constructor of the service class, initialize the list of colors with objects for red, green, and blue. Finally, create a REST API controller to manage the list of colors.
```

Copilot Proposed Change: Color.java

JAVA

```java
package com.training4programmers.demowebapi;

public class Color {
    private int id;
    private String name;
    private String hexCode;

    public Color(int id, String name, String hexCode) {
        this.id = id;
        this.name = name;
        this.hexCode = hexCode;
    }

    // getters and setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getHexCode() {
        return hexCode;
    }

    public void setHexCode(String hexCode) {
        this.hexCode = hexCode;
    }
}
```

Copilot Proposed Change: ColorService.java

JAVA

```java
package com.training4programmers.demowebapi;

import java.util.ArrayList;
import java.util.List;

public class ColorService {
    private List<Color> colors;

    public ColorService() {
        colors = new ArrayList<>();
        colors.add(new Color(1, "Red", "#FF0000"));
        colors.add(new Color(2, "Green", "#00FF00"));
        colors.add(new Color(3, "Blue", "#0000FF"));
    }

    public List<Color> getColors() {
        return colors;
    }

    public void addColor(Color color) {
        colors.add(color);
    }

    public void replaceColor(int id, Color color) {
        for (int i = 0; i < colors.size(); i++) {
            if (colors.get(i).getId() == id) {
                colors.set(i, color);
                return;
            }
        }
    }

    public void deleteColor(int id) {
        colors.removeIf(color -> color.getId() == id);
    }
}
```

Copilot Proposed Change: ColorController.java

JAVA

```java
package com.training4programmers.demowebapi;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/colors")
public class ColorController {
    private ColorService colorService;

    public ColorController() {
        this.colorService = new ColorService();
    }

    @GetMapping
    public List<Color> getColors() {
        return colorService.getColors();
    }

    @PostMapping
    public void addColor(@RequestBody Color color) {
        colorService.addColor(color);
    }

    @PutMapping("/{id}")
    public void replaceColor(@PathVariable int id, @RequestBody Color color) {
        colorService.replaceColor(id, color);
    }

    @DeleteMapping("/{id}")
    public void deleteColor(@PathVariable int id) {
        colorService.deleteColor(id);
    }
}
```

Run the code, and load the endpoint `http://localhost:8080/colors` in a web browser. You should see a JSON array of color objects.

Demo: Python Console App in VS Code

This is a demo to show how to use GitHub Copilot to help you write a new Python program. These are suggested prompts to use with GitHub Copilot. Feel free modify them or add additional prompts to get the desired results.

1. Create a "Hello, World!" Program

```text
Hello, I am new to Python programming, but I do have computer programming experience. I would like to create a simple "Hello, World!" program in Python. Can you help me?
```

2. Learn about Build Tools for Python Project

```text
Does Python support any build configuration tools? If so, what are they and how do I use them? Which one do you recommend?
```

3. Configure the Python Project to Use a Virtual Environment and PIP

```text
Please explain how to configure a Python project to use Virtual Environment and PIP.
```

4. Add Unit Testing to the Python Project.

```text
A friend of mine told me that I should add unit testing to my Python project. Can you help me with that? Which Python unit testing framework would you recommend?
```

5. Add PyTest to the Python Project.

```text
Show me how to incorporate PyTest into my Python project. Also, show me how to write a simple test case using PyTest.
```

6. Setup Source Control for the Python Project.

```text
```

```
I need to work on my project on multiple computers and track changes to my code. Is there a programming tool that I could use for this? If so, which one do you recommend?
```

7. Setup Git for the Python Project.

```
Explain to me how to setup Git for my Python project. I would like to know how to create a new repository, add files to the repository, and commit changes to the repository. Also, which files do I need to exclude from source control?
```
TEXT

Demo: C# ASP.NET MVC Web API with Visual Studio

This demo shows how to use GitHub Copilot to help you write a new C# ASP.NET MVC Web API application with Visual Studio. The prompts below are suggestions to help you explore GitHub Copilot; please feel free to modify them.

1. Install the GitHub Copilot extensions for Visual Studio. Visual Studio needs both the GitHub Copilot and GitHub Copilot Chat extensions.
2. Open Visual Studio and create a new project with the following options:
   - Project type: ASP.NET Core Web API
   - Project name: DemoWebAPI
   - Framework: .NET 8 (Long-Term Support)
   - Authentication type: None
   - Configure HTTPS: Checked
   - Enable OpenAPI support: Checked
   - Do not use top-level statements: Unchecked
   - Use controllers: Checked

Click `Create` to have Visual Studio create the project. Once created, please proceed.

3. Run the project by clicking the green arrow in the toolbar. The project will start and open a new browser window - this is the Swagger UI for the Web API project. The URL will be localhost. Visual Studio assigns a port number when it creates the project.
4. To open Copilot Chat, click on the `View` menu, then click 'GitHub Copilot Chat'.

5. Open the `Program.cs` file in the editor. Highlight the following line of code.

Program.cs

```
C#
builder.Services.AddControllers();
```

6. In the Copilot Chat window, type the following prompt and press `Enter` to see the response from Copilot.

Copilot Chat Prompt

```
TEXT
What are controllers in an ASP.NET Core Web API application?
```

7. Review the response. Notice that is detects the line of code you highlighted, and offers a response to the prompt. It explains that controllers are classes that handle incoming HTTP requests and return responses to the client. It also provides a code example of how to create a new controller. It may even offer to create a new controller for you. If you like the response, you can accept it. Remember, Copilot and generative AI is non-deterministic, so the response will vary.
8. If present, click the "Insert in new file" button, save the file in the `Controllers` folder. If the "Insert in new file" button is not present, create a new file in the `Controllers` folder and name it after the class code generated by Copilot. Copy the code from the chat window and paste it into the new file. Save the file.
9. If any namespaces are missing, use the usual Visual Studio tooling (the lightbulb) to add the missing namespaces. The final controller code should look similar to this:

Copilot Proposed Change: Controllers/MyController.cs

```C#
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("[controller]")]
public class MyController : ControllerBase
{
    [HttpGet]
    public IEnumerable<string> Get()
    {
        return new string[] { "value1", "value2" };
    }

    [HttpPost]
    public void Post([FromBody] string value)
    {
        // Code to handle POST request
    }
}
```

10. Highlight the following code.

Controllers/MyController.cs

```C#
[HttpGet]
public IEnumerable<string> Get()
{
    return new string[] { "value1", "value2" };
}
```

Prompt Copilot with the following question:

Copilot Chat Prompt

```TEXT
How do access this endpoint when I run the application?
```

The probable response will answer the question specifically for the `Get` method, but will take into

account the surrounding code such as the controller name to provide the answer.

11. With the following code still highlighted, prompt Copilot with the following question, "How do I write a unit test for this code?"

Controllers/MyController.cs

```
C#

[HttpGet]
public IEnumerable<string> Get()
{
    return new string[] { "value1", "value2" };
}
```

The probable response will be something similar to this:

Copilot Chat Response

> To write a unit test for the Get method in your MyController class, you can use a testing framework like xUnit. Here's an example of how you might write such a test:

Controllers/MyControllerTests.cs

```C#
using Xunit;
using Microsoft.AspNetCore.Mvc;
using System.Linq;
using DemoWebApi.Controllers;

public class MyControllerTests
{
    [Fact]
    public void Get_ReturnsExpectedValues()
    {
        // Arrange
        var controller = new MyController();

        // Act
        var result = controller.Get();

        // Assert
        var values = result.ToList();
        Assert.Equal(2, values.Count);
        Assert.Equal("value1", values[0]);
        Assert.Equal("value2", values[1]);
    }
}
```

Copilot Chat Response Continued

In this test:

* The Arrange step creates an instance of MyController.
* The Act step calls the Get method.
* The Assert step checks that the returned values are as expected.

This test checks the behavior of the Get method in isolation, which is the essence of unit testing. If the Get method had dependencies, you would typically use mocking to isolate the behavior of the method from its dependencies. However, in this case, the Get method has no dependencies, so no mocking is necessary.

12. In the previous response, Copilot not only provided the code for the unit test, but also explained the purpose of the test and the steps involved in writing it. To accept the response, you will need to

create a new unit test project. As you probably know, in C#, unit tests are not colocated with the application code for the same project, so let's create a new XUnit project to host the unit test code. Right-click on the solution in the Solution Explorer, and select `Add` > `New Project`. In the `Create a new project` dialog, select `xUnit Test Project` and click `Next`. Name the project `DemoWebApi.Tests` and click `Create`. Once the project is created, proceed.

13. With the new unit test project created, return to the chat and add the unit test to the project using the techniques described in the previous steps. It is recommended to create a `Controllers` folder in the unit test project and save the unit test code in a file named `MyControllerTests.cs`. Once the file is saved, proceed.

14. To run the tests, the main `DemoWebAPI` project will need to be added as a reference to the unit test project. Right-click on the `Dependencies` node in the `DemoWebApi.Tests` project, and select `Add Project Reference`. In the `Add Project Reference` dialog, select the `DemoWebAPI` project and click `OK`. Once the reference is added, proceed.

15. Open the Test Explorer window and run the tests. The tests should pass using the code above, if the code generated for you is slightly different they may or may not pass.

16. Between the `Get` and `Post` methods in the `MyController` class, add the following code:

Controllers/MyController.cs

```C#
// GET my/1
```

When you press enter after typing the comment, Copilot will suggest code to implement the endpoint described by the comment. It may make suggestions one line at a time, or it may suggest the entire method at once.

Review the suggestions made, accept and modify as needed. The final code should look similar to this:

Controllers/MyController.cs

```C#
// GET my/1
[HttpGet("{id}")]
public string Get(int id)
{
  return "value";
}
```

17. In the Copilot chat window, type the following prompt and press Enter to see the response from Copilot.

Copilot Chat Prompt

```TEXT
I need a service to manage a list of colors. Each color will have an id, a name, and a hexcode. Please generate a Color class, a service class to manage a list of colors, including adding, replacing, and deleting a color. In the constructor of the service class, initialize the list of colors with objects for red, green, and blue. Also, how do I register the service with the Demo Web API application and inject it into MyController?
```

The probable response will include explanations and code examples for the requested classes and configurations. Below are the code snippets generated when creating this tutorial.

Copilot Proposed Change: Models/Color.cs

```C#
public class Color
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string HexCode { get; set; }
}
```

Copilot Proposed Change: Services/ColorService.cs

C#

```
1   public class ColorService
2   {
3       private List<Color> _colors;
4
5       public ColorService()
6       {
7           _colors = new List<Color>
8           {
9               new Color { Id = 1, Name = "Red", HexCode = "#FF0000" },
10              new Color { Id = 2, Name = "Green", HexCode = "#00FF00" },
11              new Color { Id = 3, Name = "Blue", HexCode = "#0000FF" }
12          };
13      }
14
15      public IEnumerable<Color> GetColors() => _colors;
16
17      public Color GetColor(int id) => _colors.FirstOrDefault(c => c.Id == id);
18
19      public void AddColor(Color color) => _colors.Add(color);
20
21      public void ReplaceColor(int id, Color color)
22      {
23          var index = _colors.FindIndex(c => c.Id == id);
24          if (index != -1)
25          {
26              _colors[index] = color;
27          }
28      }
29
30      public void DeleteColor(int id)
31      {
32          var color = _colors.FirstOrDefault(c => c.Id == id);
33          if (color != null)
34          {
35              _colors.Remove(color);
36          }
37      }
38  }
```

Copilot Proposed Change: Program.cs

```C#
builder.Services.AddSingleton<ColorService>();
```

Copilot Proposed Change: Controllers/MyController.cs

```C#
public class MyController : ControllerBase
{
    private readonly ColorService _colorService;

    public MyController(ColorService colorService)
    {
        _colorService = colorService;
    }

    // Existing code...
}
```

If you like, integrate the new code into the existing application. It would be best if you renamed the `MyController` to `ColorsController`. Update the existing methods to use the service, and add two methods for replacing and deleting a color. Also, the unit tests will need to be updated. Experiment with Copilot in-editor suggestions and Copilot chat to complete the changes.

Demo: React TypeScript Web App with Visual Studio Code

This demo shows how to use GitHub Copilot to help you write a new React TypeScript Web App application with Visual Studio Code. The prompts below are suggestions to help you explore GitHub Copilot; please feel free to modify them.

1. Open Visual Studio Code. Ensure you have the GitHub Copilot extension installed. If not, you can install it from the Visual Studio Code Marketplace. Once installed, you will need to sign in with your GitHub account.
2. On the Activity Bar, click the Chat/Message looking icon. Then click on the "GitHub Copilot Chat" panel.
3. We will assume we know nothing about React or building web apps with modern JavaScript frameworks and libraries. We will assume we have some general programming experience, some TypeScript experience, and know what web programming is. We will use GitHub Copilot to help us

write a new React TypeScript Web App application.

4. Type the following prompt in the GitHub Copilot Chat panel.

Copilot Chat Prompt

```text
My manager asked me to create a new client-side web application using TypeScript to build a calculator. Are there any TypeScript libraries or frameworks that can help me build a client-side web application? Which one would you recommend? My manager told me to use something popular, with a large a community, and is component-based. I am not sure what component-based means, but that is what my manager was told me.
```

Most likely, Copilot will respond with something similar to this.

Copilot Chat Response

> Yes, there are several popular TypeScript libraries and frameworks that can help you build a client-side web application. Given your manager's requirements, I would recommend using React with TypeScript.
>
> React is a popular JavaScript library for building user interfaces, particularly for single-page applications. It's used for handling the view layer for web and mobile apps. React allows you to design simple views for each state in your application, and it will efficiently update and render the right components when your data changes. React's component-based architecture means you build your UI out of small, reusable pieces.
>
> React has a large community, which means you'll find a lot of resources, tutorials, and third-party libraries that can help you. It's maintained by Facebook and a community of individual developers and companies.

It is possible, that Copilot will recommend something other than React with TypeScript. No worries, if it does that is ok, we will still use React with TypeScript for this demo.

5. In addition to the suggestion to use React, Copilot probably gave you some example code on how to create a React component.

Copilot Chat Response

```typescript
import React, { Component } from 'react';

interface AppState {
  result: number;
}

interface AppProps {
  // define any props here
}

class Calculator extends Component<AppProps, AppState> {
  constructor(props: AppProps) {
    super(props);
    this.state = {
      result: 0,
    };
  }

  // define your methods here

  render() {
    return (
      <div>
        {/* Your UI here */}
      </div>
    );
  }
}

export default Calculator;
```

This code demo is helpful, but React requires a project configuration before we can start creating components. Let's ask Copilot about this.

Copilot Chat Prompt

```text
React sounds great. I have heard that these JavaScript/TypeScript web projects require a lot of configuration. Is there a way to create a new React TypeScript Web App project with a single command? If so, what is the command?
```

Most likely, Copilot will respond with something similar to this.

Copilot Chat Response

> Yes, there is a way to create a new React TypeScript project with a single command. You can use Create React App, which is a command line tool that scaffolds a new React project with a lot of sensible defaults and configurations out of the box.

There is a problem with this response. Create React App is no longer supported. One challenge with Copilot is that it is not always aware of the latest changes to specific tools and languages. Whenever using a tool suggested by Copilot it is best practice to verify the tool is still used and it is even better if you look for related tools. Let's see if Copilot can help us find another tool.

Copilot Chat Prompt

TEXT

> I spoke to a friend to double check whether Create React App is still the best way to create a new React TypeScript Web App project. My friend told me that Create React App is no longer supported. Are there any other tools that can help me create a new React TypeScript Web App project with a single command? If so, what is the command?

Most likely, Copilot will respond with something similar to this.

Copilot Chat Response

> Yes, there is another popular tool called Vite that you can use to create a new React TypeScript project. Vite is a build tool and development server created by Evan You, the creator of Vue.js. It offers faster and leaner development for modern web projects.

As of the writing of this demo, Vite is a popular tool for creating new React TypeScript Web App projects. Let's use Vite to create a new React TypeScript Web App project. This is a much better choice.

6. Most likely when Copilot suggested Vite, it offered some instructions on how to use it to create a React with TypeScript application. Compare the instructions to the sample instructions below.

Copilot Chat Response

TYPESCRIPT

```typescript
npx create-vite my-app --template react-ts
```

Open the terminal window in VS Code (View > Terminal) and run the command `npx create-vite my-app --template react-ts` to create a new React TypeScript Web App project Replace `my-app` with the name of our project which we will call `calc-app`. This command will create a new directory called `calc-app` and scaffold a new React TypeScript Web App project inside it.

Command Line: Create a new React TypeScript Web App project with Vite

CONSOLE

```
npx create-vite calc-app --template react-ts
```

After creating the project, follow the instructions to change into the folder and install the project dependencies with `npm install`.

Next, use VS Code to open the `calc-app` folder. You can do this by clicking on the "Open Folder" button in the welcome screen or by clicking on "File" > "Open Folder" in the menu bar.

7. Open the terminal window and run the following command to start the React application. Open a web browser, browse the URL, and view the page.

Command Line: Run the React Application

CONSOLE

```
npm run dev
```

8. Review the project structure and you will see a lot of files have been created. The most important files are the `src` folder and the `package.json` file. The `src` folder contains the source code for the React application. The `package.json` file contains the project dependencies and scripts to run the project. Do not worry about the other files for now.
9. In the `src` folder open the `main.tsx` file. It will look similar to this.

File: src/main.tsx

```typescript
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.tsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Highlight the code that calls the `ReactDOM.createRoot` method. Highlight the entire function call, all five lines.

In the Copilot Chat panel, type the following prompt.

Copilot Chat Prompt

```text
/explain What is this code doing? Also, what is this odd-looking HTML syntax mixed with the TypeScript code? I thought TypeScript was a superset of JavaScript, not HTML.
```

Most likely, Copilot will respond with something similar to this.

Copilot Chat Response

> This code is part of a React application written in TypeScript. It's responsible for rendering the main App component into a DOM element with the id 'root'.
>
> The HTML-like syntax you see is called JSX (JavaScript XML). It's a syntax extension for JavaScript, used by libraries like React, to describe what the UI should look like. TypeScript supports JSX syntax as well, hence you see it in this TypeScript file.

Copilot explains the React library and explains the JSX syntax. Copilot does a lot more than merely suggest code. It can explain programming concepts, developer tools, programming languages, and the specifics of libraries and frameworks.

9. Let’s create new React component to serve as the calculator. In the `src` folder, create a new file

called `Calculator.tsx`. Click the stars (`<CTRL>+i` or `<CMD>+i`) to display an in-editor file tab Copilot dialog. Type the following prompt in the Copilot Chat panel.

Copilot Chat Prompt

```text
Define a component that represents a calculator. It should have a button for each digit, a button for each operator (+, -, *, /), and a display for the result.
```

The response from Copilot will be similar to this.

Copilot Chat Response

TYPESCRIPT

```typescript
import React, { useState } from 'react';

const Calculator: React.FC = () => {
  const [displayValue, setDisplayValue] = useState('');

  const handleButtonClick = (value: string) => {
    setDisplayValue(displayValue + value);
  };

  const handleCalculate = () => {
    try {
      const result = eval(displayValue);
      setDisplayValue(result.toString());
    } catch (error) {
      setDisplayValue('Error');
    }
  };

  const clearDisplay = () => {
    setDisplayValue('');
  };

  return (
    <div>
      <input type="text" value={displayValue} readOnly />
      <div>
        <button onClick={() => handleButtonClick('1')}>1</button>
        <button onClick={() => handleButtonClick('2')}>2</button>
        <button onClick={() => handleButtonClick('3')}>3</button>
        <button onClick={() => handleButtonClick('+')}>+</button>
      </div>
      <div>
        <button onClick={() => handleButtonClick('4')}>4</button>
        <button onClick={() => handleButtonClick('5')}>5</button>
        <button onClick={() => handleButtonClick('6')}>6</button>
        <button onClick={() => handleButtonClick('-')}>-</button>
      </div>
      <div>
        <button onClick={() => handleButtonClick('7')}>7</button>
        <button onClick={() => handleButtonClick('8')}>8</button>
        <button onClick={() => handleButtonClick('9')}>9</button>
        <button onClick={() => handleButtonClick('*')}>*</button>
```

```
      </div>
      <div>
        <button onClick={() => handleButtonClick('0')}>0</button>
        <button onClick={() => handleCalculate()}>=</button>
        <button onClick={() => clearDisplay()}>C</button>
        <button onClick={() => handleButtonClick('/')}>/</button>
      </div>
    </div>
  );
};

export default Calculator;
```

10. Open the `App.tsx` file and import the `Calculator` component. Then use the `Calculator` component in the `App` component. The `App.tsx` file will look similar to this. If you are not use how to import the `Calculator` component, ask Copilot for help.

Screenshot of Asking Copilot for Help

Screenshot of Copilot Suggested Change

Copilot attempted to figure where to place the Calculator component within the current content structure of the App component. This may or may not be what you want. Let's ask to replace ALL of the content.

Copilot Chat Prompt

```text
Please replace ALL the JSX in the App Component with a call to the Calculator component. The only JSX element in the App Component should be the Calculator. Also, please move the import statement for the Calculator component to the top of the file.
```

11. Accept the change offered by Copilot. Clean up any extra code from the original source. The final `App.tsx` file will look similar to this.

File: src/App.tsx

TYPESCRIPT

```typescript
import './App.css'
import Calculator from './Calculator'

function App() {
  return (
    <>
      <Calculator />
    </>
  )
}

export default App
```

```
function App() {
  const [count, setCount] = useState(0)
```

How do I replace this content with the Calculator component?

Accept Discard 2 changes

```
  import Calculator from './Calculator'

  return (
    <>
      <div>
        <a href="https://vitejs.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div class="card">
        <Calculator />
      </div>
      <p class="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}
```

The web page should automatically refresh and you should see the calculator. You can now use the calculator to perform simple arithmetic operations.

12. The calculator works, but there is a problem. There is a function call to `eval` in the `Calculator` component. The `eval` function is a security risk and should not be used. Let's ask Copilot for help to fix this.

In the `Calculator.tsx` file, highlight the `eval` function call, right-click, and select `Fix This` under the `Copilot` submenu. Add the following text to the `/fix` Copilot command.

Copilot Chat Prompt

TEXT

```
/fix Please replace the call to the eval function with a safer alternative. Is there a safer third-party NPM package that could help me with this?
```

You may get a suggestion like this:

Screenshot of Copilot MathJS Suggestion

```
function App() {
  const [count, setCount] = useState(0)

  How do I replace this content with the Calculator component?
  Accept  Discard                                          2 changes

  import Calculator from './Calculator'

  return (
    <>
      <div>
        <a href="https://vitejs.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div class="card">
        <Calculator />
      </div>
      <p class="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}
```

Accept the solution and clean up the file. Make sure imports are at the top and smal fixes. Remember, we do have some TypeScript experience. We can make sure the code is correct.

How do we install the MathJS package? Ask Copilot for help.

Stop the web application, `<CTRL+C>` in the terminal. Ensure the terminal panel is still open. Then type the following prompt in the Copilot Chat panel.

Copilot Chat Prompt

```text
@terminal How do I install the MathJS NPM package?
```

Most likely, Copilot will respond with something similar to this.

Copilot Chat Response

```
npm install mathjs
```

Click the Terminal Copy icon in the chat, and run the command.

14. Once the package is installed, start the web app again.

Command Line: Run the React Application

```
npm run dev
```

In the web browser, verify the new calculator solution works.

The final `Calculator.tsx` file will look similar to this.

File: src/Calculator.tsx

TYPESCRIPT

```typescript
import React, { useState } from 'react';
import { evaluate } from 'mathjs';

const Calculator: React.FC = () => {
  const [displayValue, setDisplayValue] = useState('');

  const handleButtonClick = (value: string) => {
    setDisplayValue(displayValue + value);
  };


  const handleCalculate = () => {
    try {
      const result = evaluate(displayValue);
      setDisplayValue(result.toString());
    } catch (error) {
      setDisplayValue('Error');
    }
  };

  const clearDisplay = () => {
    setDisplayValue('');
  };

  return (
    <div>
      <input type="text" value={displayValue} readOnly />
      <div>
        <button onClick={() => handleButtonClick('1')}>1</button>
        <button onClick={() => handleButtonClick('2')}>2</button>
        <button onClick={() => handleButtonClick('3')}>3</button>
        <button onClick={() => handleButtonClick('+')}>+</button>
      </div>
      <div>
        <button onClick={() => handleButtonClick('4')}>4</button>
        <button onClick={() => handleButtonClick('5')}>5</button>
        <button onClick={() => handleButtonClick('6')}>6</button>
        <button onClick={() => handleButtonClick('-')}>-</button>
      </div>
      <div>
        <button onClick={() => handleButtonClick('7')}>7</button>
        <button onClick={() => handleButtonClick('8')}>8</button>
```

```
      <button onClick={() => handleButtonClick('9')}>9</button>
      <button onClick={() => handleButtonClick('*')}>*</button>
    </div>
    <div>
      <button onClick={() => handleButtonClick('0')}>0</button>
      <button onClick={() => handleCalculate()}>=</button>
      <button onClick={() => clearDisplay()}>C</button>
      <button onClick={() => handleButtonClick('/')}>/</button>
    </div>
  </div>
 );
};

export default Calculator;
```

This concludes the demo. You have created a new React TypeScript Web App project with a calculator component. You used GitHub Copilot to help you write and understand the code. You also used GitHub Copilot to help you find a safer alternative to the `eval` function. You have also learned how to use GitHub Copilot to help you with the terminal commands. You can now use this project as a starting point for your own React TypeScript Web App projects. You can also use GitHub Copilot to help you with other programming tasks and to learn new programming concepts. GitHub Copilot is a powerful tool that can help you become a better programmer.

If you want to dive deeper, use Copilot to help you with the following tasks:

- Explain the `useState` hook in React.
- Enhance the application to show a history of calculations.
- Add additional math operations.

Module 2: Copilot Suggestions

While Copilot is supported by many editors, the most popular editor is Visual Studio Code. The following notes will help you start with GitHub Copilot in Visual Studio Code. Below are some notes on how to use Copilot in Visual Studio Code. If using another editor, refer to the official instructions at the links above.

Practical Copilot Techniques for Visual Studio Code

- Open a source file, write a comment description of what you want, type `<ENTER>`, and a suggestion should appear on the following line.
  - If you like the suggestion, hit `<TAB>` on the keyboard to accept.
  - If the suggestion is unacceptable and you want to type something else, type `<ESC>` on the keyboard to decline the suggestion. Then, you may continue typing.
  - If no suggestion appears, or the suggestion disappears, and you want it back, then hit `<ALT>+\` on the keyboard; if there is a suggestion, it will appear.
  - When there are multiple suggestions, to view them, type `<ALT>+]` or `<ALT>+[` to cycle left and right through them. Also, you can hover over the suggestion with your mouse cursor to see more.
  - To view multiple suggestions in a new editor tab, type `<CTRL>+<ENTER>` to open a new tab and review the generated suggestions.

Useful Copilot Chat Techniques for Visual Studio Code

- To open a chat session to discuss a code snippet, highlight the code in the editor and type `<CTRL>+i` on Windows and Linux and `<CMD>+i` on macOS.
- If you do not understand some lines of code, highlight them, right-click, and select `Copilot` → `Explain This` from the context menu.
- If you want to fix some lines of code, highlight them, right-click, and choose `Copilot` → `Fix This` from the context menu.
- To get help within Copilot Chat, open the chat panel, type in `/help`, and then hit `<ENTER>`. A good summary of available Copilot Chat commands will appear.
- `@<some scope>` handles your prompts within a scope; there are three scopes: `@workspace`, `@vscode`, and `@terminal`
  - `@workspace` is for creating new projects and managing existing projects.

- `@vscode` is for editor configuration and managing extensions.
- `@terminal` is for running commands in the terminal.

Explore Non-Deterministic Code Suggestions

Deterministic and non-deterministic are terms used to describe the behavior of systems, including computer programs.

A deterministic program is one where, for a given input, the program's output and state will always be the same, no matter how many times you run it. This predictability can make deterministic programs easier to test and debug.

On the other hand, a non-deterministic program is one where the program's output or state can change even with the same input. The varying output could be due to random number generation, system time, concurrent execution, or external system interactions.

As for GitHub Copilot, OpenAI Codex powers Copilot; OpenAI Codex is a model trained on various public sources, including code repositories. GitHub Copilot provides non-deterministic suggestions. While GitHub designed Copilot to provide relevant code suggestions based on the context of your code, the exact suggestions can vary even in similar contexts. The variation is due to the underlying machine learning model, which has a degree of randomness in its outputs.

Demo: C++ Code Suggestions

The following demo will explore how to use the in-editor code suggestions provided by GitHub Copilot

1. Create a function that adds three numbers together. In the editor, type in the following comment.

```cpp
// add three numbers together
```

When you hit enter, a code suggestion will appear at the start of the next line. Most likely, the suggestion will be a function that adds three numbers together. If you like the suggestion, hit tab to accept it. If you do not like the suggestion, hit escape to decline it. If you want to see the suggestion again, hit alt + backtick. If there are multiple suggestions, you can cycle through them with `<ALT>+]` and `<ALT>+[`.

2. To view many suggestions in a new editor tab, type ctrl + enter to open a new tab and review the generated suggestions.

3. Above the code add the comment:

```cpp
int addTwoNumbers(int a, int b) {
   return a + b;
}
```

4. Place the cursor on the line following the original comment, type `<ALT>+\` to generate a suggestion. Observe how the suggestions follows the coding example of the `addTwoNumbers` function.

5. Revise the `add_two_numbers` function to look like this.

```cpp
int add_two_numbers(int a, int b) {
   return a + b;
}
```

6. Place the cursor on the line following the original comment, type `<ALT>+\` to generate a suggestion. Observe how the suggestions follows the coding example of the `add_two_numbers` function.

7. Delete the `add_two_numbers` function and any code and comments for the add three numbers function.

8. Update the code in the file with the code below. Instructions are in all caps. Do not type instructions into the file. After each comment, accept the suggestion.

```cpp
#include <iostream>

// TYPE THE COMMENT BELOW AND GET SUGGESTION
// generate an array of random integers


// TYPE THIS COMMENT BELOW AND GET SUGGESTION
// calculate some descriptive statistics for an array of integers

int main() {
   // DO NOT TYPE THIS, PLACE CURSOR HERE AND WAIT FOR SUGGESTION
   return 0;
}
```

9. Compile and run the program to see the output.

Demo: Java Code Suggestions

The following demo will explore how to use the in-editor code suggestions provided by GitHub Copilot. Create a new "Hello, World!" style Java console project. Open the project in IntelliJ IDEA.

1. Create a function that adds two numbers together. In the editor, type in the following comment.

```java
// add two numbers together
```

2. Type <TAB> to accept the suggestion. Observe how the suggestion follows the coding example of the addTwoNumbers function.

```java
private static int addTwoNumbers(int firstNumber, int secondNumber) {
    return firstNumber + secondNumber;
}
```

3. Delete the accepted code, place cursor at end of the original comment, then hit <ENTER>. On the IntelliJ IDEA menu, click Tools > Copilot > Show Completions. Review the suggestions that appear. Accept one of the suggestions, if you like.

Module 3: VS Code Copilot Scopes

Visual Studio Code Copilot has three scopes: `@workspace`, `@vscode`, and `@terminal`. Each scope has its own set of commands and prompts. The following notes will help you understand how to use each scope. In this module, we will explore the `@vscode` and `@terminal` scopes. We explore the `@workspace` scope in module 5.

VS Code Copilot Prompts

1. Configure Tabs, Spaces, and Forced Indenting

Copilot Chat Prompt

```text
@vscode how do I set the tab size and ensure spaces are used instead of tabs? also, how do I force this on every file?
```

1. Disable Copilot for Markdown Files

Copilot Chat Prompt

```text
@vscode how do I disable copilot for markdown files?
```

1. Which Extensions to Configure CMake Files?

Copilot Chat Prompt

```text
@vscode which extension do I use to configure cmake files?
```

Terminal Copilot Chat

- Delete a Folder and Its Contents

Copilot Chat Prompt

```text
@terminal How do I delete a folder and all its contents?
```

Terminal Copilot & Containerization

1. Getting Started with Containerization

Copilot Chat Prompt

```text
@terminal I am new to containerization. Please explain the basics of containerization to me. What tools can I use? Which tool do you recommend?
```

2. Getting Started with Docker

Copilot Chat Prompt

```text
@terminal How do I get started with Docker? Is there such a thing as a "Hello, World!" with Docker technology?
```

3. Create a Docker Image for a Spring Boot Application

Copilot Chat Prompt

```text
@terminal I am a Java Spring Boot programmer. My manager told me I needed to deploy my latest Spring Boot application to the cloud using a Docker image. Can you explain how to create a Docker image for a standard Spring Boot application?
```

4. Use Docker to Build the Application and Create the Deploy Image

Copilot Chat Prompt

```text
@terminal This is great! Thank you! One more thing: my manager said I need to use Docker to build and deploy the application. He mentioned something about a multi-stage Dockerfile containing a build container or image that builds the JAR file and then copies the built JAR file to the image used for deployment. Can you help me with this?
```

5. Deploy the Docker Image to a Container Registry within AWS

Copilot Chat Prompt

```text
@terminal Finally, how do I push my final image to a container registry on AWS?
```

6. Access the Containers running on Kubernetes.

Copilot Chat Prompt

```text
```

```
@terminal The ops team told me my image was used to deploy containers within a Kubernetes cluster. What is Kubernetes? Also, they said I can view my running application with some kind of command line tool. What is the command line tool used to administer Kubernetes? How do I access the cluster with it? Kubernetes is running on AWS.
```

7. What are Kubernetes Pods?

Copilot Chat Prompt

```TEXT
@terminal Explain to me what Kubernetes pods are. How do I view their health?
```

8. Deploy my Spring Boot App to Kubernetes

Copilot Chat Prompt

```TEXT
@terminal The ops team told me they have set up a Kubernetes cluster for testing and development. I must deploy my Spring Boot Application from the Container Registry to this new cluster. Also, my application needs a connection string environment variable to connect to the data when running in the cluster. How do I securely store the connection string in AWS and make it an environment variable on my running pods?
```

9. Kubectl to Perform Deployment

Copilot Chat Prompt

```TEXT
@terminal Thanks! This looks great. One last thing: how do I deploy this with kubectl?
```

Module 4: Prompts and Suggestions

This module goes deeper into how prompts and suggestions work in GitHub Copilot. Also, we will explore additional aspects of prompt engineering.

- What data does Copilot send in the prompts to the GitHub Copilot servers?
- Copilot sends the code and comments that you are editing and the relevant surrounding context. The surrounding context includes the current file and other files opened in the editor.
- What data is retained by GitHub?
- GitHub deletes prompt data after making the suggestion, and GitHub does not store the suggestion after making it. GitHub stores user interaction data for 24 months. GitHub uses this data to improve the Copilot service, not the training AI model. GitHub trains the generative AI model with public source code, not code from private repositories.

C++ X-Shot Prompting

Zero Shot Prompting

Chat with Copilot using the following prompt.

Copilot Chat Prompt

```text
Create a new C++ class for an Airplane.
```

Multiple Shot Prompting

Chat with Copilot using the following prompt which includes an example of what we are looking for.

Copilot Chat Prompt

```text
```

```
Create a new C++ class for an Airplane.

Use the following code as an example:

class Car {
  private:
  string make_name;
  string model_name;
  string registration_number;

  public:
  string getMakeName();
  string getModelName();
  string getRegistrationNumber();
  void drive();
}
```

Java X-Shot Prompting

Zero Shot Prompting

Chat with Copilot using the following prompt.

Copilot Chat Prompt

```TEXT
Create a new Java class for an Airplane.
```

Multiple Shot Prompting

Chat with Copilot using the following prompt which includes an example of what we are looking for.

Copilot Chat Prompt

```TEXT
```

```
Create a new Java class for an Airplane.

Use the following C++ code as an example for the Java class:

class Car {
  private:
  string make_name;
  string model_name;
  string registration_number;

  public:
  string getMakeName();
  string getModelName();
  string getRegistrationNumber();
  void drive();
}
```

Python X-Shot Prompting

Zero Shot Prompting

Chat with Copilot using the following prompt.

Copilot Chat Prompt

```text
Create a new Python class for an Airplane.
```

Multiple Shot Prompting

Chat with Copilot using the following prompt which includes an example of what we are looking for.

Copilot Chat Prompt

```text
```

Create a new Python class for an Airplane.

Use the following C++ code as an example for the Python class:

```
class Car {
  private:
  string make_name;
  string model_name;
  string registration_number;

  public:
  string getMakeName();
  string getModelName();
  string getRegistrationNumber();
  void drive();
}
```

Module 5: Copilot and Workspaces

The `@workspace` scope is for creating new projects and managing existing projects. The following demonstrations will help you understand how to use the `@workspace` scope. We explored the `@vscode` and `@terminal` scopes module 3. The `@workspace` scope is the primary scope for interacting with source code and Copilot Chat. Only the code in open editor windows is available to Copilot when working with existing code. Copilot does not have access to the file system except for creating a new project with the `@workspace/new` command. Copilot does not have access to your entire solution or other code repositories when making suggestions via Copilot.

Workspace New Command for a C++ Project

The following prompts demonstrate how to create a new project with GitHub Copilot and its `@workspace/new` command. There are several iterations of the same prompt adding additional detail to the new project request to explore how far GitHub Copilot can go to create a great start to a new project!

To run these commands, open the GitHub Copilot in Visual Studio Code.

- Iteration 1 - Simply Create a C++ Project

```text
@workspace /new Create a new C++ project.
```

- Iteration 2 - Create a C++ Project using a Specific Build Tool

```text
@workspace /new Create a new C++ project with CMake.
```

- Iteration 3 - Create a C++ Project using Specific Versions

```text
@workspace /new Create a new C++ project with CMake. My C++ version is 23. My CMake is version 3.28.2.
```

- Iteration 4 - Add Explicit Support for Google Test

```text
```

```
@workspace /new Create a new C++ project with Google Test and Cmake. The C++ version is 23. CMake is version 3.28.2. The CMake configuration will download the Google Test source from GitHub with tagged with v1.14.0.
```

- Iteration 5 - Add Documentation

```TEXT
@workspace /new Create a new C++ project with Google Test and Cmake. The C++ version is 23. CMake is version 3.28.2. The CMake configuration will download the Google Test source from GitHub with tagged with v1.14.0. Also, give me instructions in a README.md on how to build the project with CMake not Make. Also, in the README.md, explain to me how to run the executable for the application and the tests.
```

Workspace Scoped Prompts for a C++ Project

When chatting with GitHub Copilot it is possible to chat about your project and get specific answers. The key is context. When chatting with Copilot using no command in the prompt, the chat will be about the open files in your editor. If you wish to include all files in your project, the `@workspace` command must be prepended to the prompt.

Generating a C++ Project

The following iteration of prompts further refines the project creation instructions until the instructions match what we are looking for. Prompting is kind of like programming, but with human language.

- Iteration 1: Create a new C++ Project with a CMake Configuration

```TEXT
@workspace create a new cpp project with a cmake configuration
```

- Iteration 2: Create a new C++ Project with a CMake Configuration including Google Test

```TEXT
@workspace create a new cpp project with a cmake configuration, include support for unit tests with Google Test
```

- Iteration 3: Create a new C++ Project with Specific Specifications

```TEXT
```

```
@workspace Create a new cpp project with a cmake configuration and include support for unit tests with Google Test. The CMake version will be 3.28. The C++ version will be 23. The Google Test tag will be v14. Please include instructions on how to build the project with cmake instead of make. Also, please generate the appropriate .gitignore file, be sure to include .DS_Store for support on macOS.
```

Add a Person Class

The following iteration of prompts further refines the adding of a `Person` class instructions until the instructions match what we are looking for. This prompt usually requires working CMake configuration and `build` folder.

- Iteration 1: Create a new Person Class

```TEXT
@workspace add a person class file
```

- Iteration 2: Create a new Person Class with specific Fields

```TEXT
@workspace Add a person class in a new file, the person will have the following fields: first name, last name, age, street address, city, state, and zip code.
```

- Iteration 3: Create a new Person Class with specific Fields and Methods

```TEXT
@workspace Add a person class in a new file, the person will have the following fields: first name, last name, age, street address, city, state, and zip code. Make each field private with public get and set functions. Add a method that returns the full name of the person.
```

- Iteration 4: Create a new Person Class with Unit Tests

```TEXT
@workspace Add a person class in a new file, the person will have the following fields: first name, last name, age, street address, city, state, and zip code. Make each field private with public get and set functions. Add a method that returns the full name of the person. Please generate unit tests for all methods.
```

- Iteration 5: Create a new Person Class with Unit Tests and Add to CMake

```TEXT
```

```
@workspace Add a person class. Implement the class using a header file and a cpp file. The person will have the following fields: first name, last name, age, street address, city, state, and zip code. Make each field private with public get and set functions. Add a method that returns the full name of the person. Please generate unit tests for all methods. Please show me how to add the new class and unit tests to my CMake configuration.
```

Extract a New Address Class From Person Class

In this example, we will not iterate on each prompt; instead we will use a series a prompts to complete a more complex refactoring task.

- Prompt 1: Extract a new class named Address

```text
@workspace Using the address related fields from the Person class, create a new Address class. Implement the Address class with a header and cpp file. Follow the pattern of the Person class for generating fields and methods.
```

- Prompt 2: Update the Person class to use the Address class.

```text
@workspace Replace the address-related fields in the Person class with the Address class. Update the Person class so the Address is created when the Person is create. Also, enable the Address to modified through a Person object.
```

Workspace New Command for a Java Project

The following prompts demonstrate how to create a new project with GitHub Copilot and its `@workspace/new` command. There are several iterations of the same prompt adding additional detail to the new project request to explore how far GitHub Copilot can go to create a great start to a new project!

To run these commands, open the GitHub Copilot in Visual Studio Code.

- Iteration 1 - Simply Create a Java Project

```text
@workspace /new Create a new Java project.
```

- Iteration 2 - Create a simple Java Project

```text
@workspace /new Create a simple Java project. Do not include unit tests, do not create extra classes, and do not use a build tool like Maven or Gradle.
```

- Iteration 3 - Create a simple Java Project using Maven as the Build Tool.

```text
@workspace /new Create a simple Java project. Do not include unit tests and do not create extra classes. Configure the project with the Maven build tool.
```

- Iteration 4 - Create a simple Java Project with Unit Test

```text
@workspace /new Create a simple Java project. Do not create extra classes. Configure the project with the Maven build tool. Add a unit test using JUnit.
```

Workspace New Command for a Python Project

The following prompts demonstrate how to create a new project with GitHub Copilot and its `@workspace/new` command. There are several iterations of the same prompt adding additional detail to the new project request to explore how far GitHub Copilot can go to create a great start to a new project!

To run these commands, open the GitHub Copilot in Visual Studio Code.

- Iteration 1 - Simply Create a Python Project

```text
@workspace /new Create a new Python project.
```

- Iteration 2 - Create a simple Python Project

```text
@workspace /new Create a new Python project do not include unit tests, requirements.txt, or a virtual environment configuration.
```

- Iteration 3 - Create a simple Python Project including Logging

```text
```

```
@workspace /new Create a new Python project do not include unit tests, requirements.txt, or a virtual environment configuration. The project should include the Python standard library logger.
```

- Iteration 4 - Create a simple Python Project including Logging and Unit Tests

```TEXT
@workspace /new Create a new Python project do not include requirements.txt and a virtual environment configuration. The project should include the Python standard library logger. Also, include unit tests using using the unittest standard library module.
```

- Iteration 5 - Create a simple Python Project including Logging, Unit Tests, and Argument Parsing

```TEXT
@workspace /new Create a new Python project do not include requirements.txt and a virtual environment configuration. The project should include the Python standard library logger. Include unit tests using using the unittest standard library module. Include argument parsing using the argparse standard library module. The program should support an argument for verbose logging and loading a configuration file using the INI format.
```

Module 6: Discuss Code with Copilot

We designed the following demonstrations to help you improve your code using GitHub Copilot. Within the module are two sections: Discuss Code and Fix Code. The Discuss Code section is for discussing code with Copilot and getting coding advice. The Fix Code section explores fixing code with Copilot. The fixes include static code errors and code patterns that lead to possible runtime errors.

C++: Advice on Getters/Setters

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Person {
  private:
    string name;
    int age;
  public:
    Person(string name, int age) {
      this->name = name;
      this->age = age;
    }
    string getName() {
      return name;
    }
    int getAge() {
      return age;
    }
    void setName(string name) {
      this->name = name;
    }
    void setAge(int age) {
      this->age = age;
    }
    string print() {
      return "Name: " + name + ", Age: " + to_string(age);
    }
};

int main() {
  Person person1("John", 30);
  cout << person1.print() << endl;
  return 0;
}
```

Compile and run the code, it should work.

2. Highlight the end `Person` class. Open the Copilot chat window. Send Copilot Chat the following

prompt.

```
@workspace /fix is using getters and setters a good coding style?
```

3. Copilot Chat can be used to get general feedback on our code. It will offer its opinion on how the code is implemented including coding styles as well as suggest improvements.

For example, a probable response from Copilot Chat would be:

The use of getters and setters (also known as accessors and mutators) is a common practice in object-oriented programming. They provide a way to control how a class's fields are accessed and modified, which can help to maintain the integrity of the data.

However, whether or not to use getters and setters can depend on the specific situation. If a field should not be directly accessible or modifiable, then using getters and setters is a good idea. On the other hand, if a field should be freely accessible and modifiable, then getters and setters may not be necessary.

In your `Person` class, the use of getters and setters seems appropriate. The fields `name` and `age` are private, so they can't be accessed or modified directly from outside the class. The getters and setters provide controlled access to these fields.

Here's your code with a minor improvement. I've added `const` to the getters to indicate that they don't modify the object:

```cpp
class Person {
private:
    std::string name;
    int age;
public:
    Person(std::string name, int age) : name(name), age(age) {}

    std::string getName() const {
        return name;
    }

    int getAge() const {
        return age;
    }

    void setName(std::string name) {
        this->name = name;
    }

    void setAge(int age) {
        this->age = age;
    }

    std::string print() const {
        return "Name: " + name + ", Age: " + std::to_string(age);
    }
};
```

In this version, I've also used the constructor initializer list to initialize `name` and `age`, which is a more idiomatic way to initialize fields in C++.

4. If the suggested improvements make sense, update your code. The final solution should look similar to this.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Person {
private:
    string name;
    int age;
public:
    Person(string name, int age) : name(name), age(age) {}

    string getName() const {
        return name;
    }

    int getAge() const {
        return age;
    }

    void setName(string name) {
        this->name = name;
    }

    void setAge(int age) {
        this->age = age;
    }

    string print() const {
        return "Name: " + name + ", Age: " + to_string(age);
    }
};

int main() {
  Person person1("John", 30);
  cout << person1.print() << endl;
  return 0;
}
```

Compile and run the code. It should work.

Java: Equivalent of C# Like Properties

Does Java Support C# styles Class Properties?
(https://stackoverflow.com/questions/2701077/does-java-have-properties-that-work-the-same-way-properties-work-in-c)

To run the following code, please create a "Hello World" style Java project using Visual Studio Code or JetBrains IntelliJ IDEA.

1. Open the Java file with the `public static void main` function. Update the code in the `Main` class with this code.

Main.java

```java
class Person {
    private String Name { get; set; };
    private int Age { get; set; };

    public Person(String name, int age) {
        Name = name;
        Age = age;
    }
}

public class Main {
    public static void main(String[] args) {

        Person person = new Person("John", 25);

        System.out.println(person.Name);
        System.out.println(person.Age);
    }
}
```

The code should fail to compile. Why? Well the code is attempting to use C# style properties in Java. Java does not support C# style properties. Instead, Java uses getter and setter methods to access and modify class properties. Can Copilot detect the bug, and offer a fix?

2. Highlight the code in the `main` function, right click, mouse over Copilot, and ask Copilot to fix the

code in the context menu.

Copilot Chat Prompt

```TEXT
/fix I am a C# developer using Java. I know C# and Java are similar. So I have tried to use C# class properties but this does not appear to work in Java. What is the correct alternative in Java that accomplishes the same goal?
```

3. Copilot should offer a fix similar to this:

Copilot Proposed Change: Main.java

```java
class Person {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}

public class Main {
    public static void main(String[] args) {
        Person person = new Person();
        person.setName("John");
        person.setAge(25);
        System.out.println(person.getName());
        System.out.println(person.getAge());
    }
}
```

4. Accept the fix and run the code. The code should run without errors.

Java: Coding Conventions

To run the following code, please create a "Hello World" style Java project using Visual Studio Code or JetBrains IntelliJ IDEA.

1. Open the Java file with the `public static void main` function. Update the code in the `Main` class with this code. Rename the `Main.java` file to `main.java` to compile and run this code.

main.java

```java
public class main {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        int z = x + y;
        System.out.println("The sum is: " + z);
    }
}
```

2. Highlight the code in the `main` class, right-click, mouse over Copilot, and ask Copilot to fix the code in the context menu.

Copilot Chat Prompt

```text
/fix Please update the code to use common Java coding conventions. As part of the refactoring, please configure constants, refactor code into methods, and use meaningful variable names.
```

3. Copilot should offer a fix similar to this:

Copilot Proposed Change: Main.java

```java
public class Main {
    public static void main(String[] args) {
        printSumOfNumbers();
    }

    private static void printSumOfNumbers() {
        final int FIRST_NUMBER = 5;
        final int SECOND_NUMBER = 10;
        int sum = addNumbers(FIRST_NUMBER, SECOND_NUMBER);
        System.out.println("The sum is: " + sum);
    }

    private static int addNumbers(int firstNumber, int secondNumber) {
        return firstNumber + secondNumber;
    }
}
```

4. Accept the fix and run the code. The code should run without errors.

Python: Improve Loop Efficiency

To run the following code, please create a "Hello World" style Python project using Visual Studio or JetBrain's Pycharm.

1. Open the Python file with the `main` function. Replace the code in the file, with this code.

```python
def main():

    nums = [1, 2, 3, 4, 5]
    double_nums = []

    for num in nums:
      double_nums.append(num * 2)

    print(nums)
    print(double_nums)


if __name__ == "__main__":
    main()
```

The code will run with no problems. But is this the most efficient way to double the numbers in the list? Copilot can help us improve the code.

2. Highlight the code, right click, mouse over Copilot, and ask Copilot to fix the code in the context menu. Add the following text following the `/fix` command:

```text
/fix improve the code so it is more efficient and more Pythonic
```

3. Copilot should offer a fix similar to this:

```text
To improve the code, I would suggest using a list comprehension instead of a for loop to create the double_nums list.
```

```python
def main():

    nums = [1, 2, 3, 4, 5]
    double_nums = [num * 2 for num in nums]

    print(nums)
    print(double_nums)


if __name__ == "__main__":
    main()
```

4. Accept the fix and run the code. The code should run without errors.

C#: Pattern Matching

To run the following code, please create a C# console application using .NET 8, with top-level statements enabled.

1. Replace the code in the `Program.cs` file with the following code:

Program.cs

C#

```
Shape shape = new Circle { Radius = 5 };
// Shape shape = new Rectangle { Width = 10, Height = 20 };
// Shape shape = new Triangle { Base = 10, Height = 20 };

if (shape is Circle)
{
  Circle circle = (Circle)shape;
  Console.WriteLine($"Processing a circle with radius {circle.Radius}");
}
else if (shape is Rectangle)
{
  Rectangle rectangle = (Rectangle)shape;
  Console.WriteLine($"Processing a rectangle with width {rectangle.Width} and height
{rectangle.Height}");
}
else if (shape is Triangle)
{
  Triangle triangle = (Triangle)shape;
  Console.WriteLine($"Processing a triangle with base {triangle.Base} and height
{triangle.Height}");
}
else
{
  Console.WriteLine("Unknown shape type");
}

interface Shape
{
  public void Draw();
}

class Circle : Shape
{
  public int Radius { get; set; }
  public void Draw()
  {
    Console.WriteLine("Drawing a circle");
  }
}

class Rectangle : Shape
{
```

```
  public int Width { get; set; }
  public int Height { get; set; }
  public void Draw()
  {
    Console.WriteLine("Drawing a rectangle");
  }
}

class Triangle : Shape
{
  public int Base { get; set; }
  public int Height { get; set; }
  public void Draw()
  {
    Console.WriteLine("Drawing a triangle");
  }
}
```

Run the code, it should work as expected.

2. Is this the best way to write this code using modern C#? Why or why not?

3. Highlight the following code block.

Program.cs

```C#
if (shape is Circle)
{
  Circle circle = (Circle)shape;
  Console.WriteLine($"Processing a circle with radius {circle.Radius}");
}
else if (shape is Rectangle)
{
  Rectangle rectangle = (Rectangle)shape;
  Console.WriteLine($"Processing a rectangle with width {rectangle.Width} and height
{rectangle.Height}");
}
else if (shape is Triangle)
{
  Triangle triangle = (Triangle)shape;
  Console.WriteLine($"Processing a triangle with base {triangle.Base} and height
{triangle.Height}");
}
else
{
  Console.WriteLine("Unknown shape type");
}
```

Once the code is highlighted in the Visual Studio editor, right-click, and select Ask Copilot from the content menu. In the dialog that appears, ask the following question:

Copilot Chat Prompt

```TEXT
/fix Is this the best way to write this code?
```

Copilot Proposed Change

```C#
if (shape is Circle circle)
{
  Console.WriteLine($"Processing a circle with radius {circle.Radius}");
}
else if (shape is Rectangle rectangle)
{
  Console.WriteLine($"Processing a rectangle with width {rectangle.Width} and height
{rectangle.Height}");
}
else if (shape is Triangle triangle)
{
  Console.WriteLine($"Processing a triangle with base {triangle.Base} and height
{triangle.Height}");
}
else
{
  Console.WriteLine("Unknown shape type");
}
```

You will notice the lines of code casting the object from one type to another for each code branch have been removed and replaced with newer pattern-matching syntax in the `if` statement conditional expression. The enhanced conditional expression is a more modern way to write the code and is more readable.

Can this be further improved? Accept the suggested change, and we will continue the discussion.

4. Highlight the same code block as before, and open the Copilot chat window.

Program.cs

```C#
if (shape is Circle circle)
{
  Console.WriteLine($"Processing a circle with radius {circle.Radius}");
}
else if (shape is Rectangle rectangle)
{
  Console.WriteLine($"Processing a rectangle with width {rectangle.Width} and height
{rectangle.Height}");
}
else if (shape is Triangle triangle)
{
  Console.WriteLine($"Processing a triangle with base {triangle.Base} and height
{triangle.Height}");
}
else
{
  Console.WriteLine("Unknown shape type");
}
```

In the chat window, ask the following question:

Copilot Chat Prompt

```TEXT
/fix Is an if-else chain the best way to structure this? Does pattern matching support a cleaner control flow structure?
```

Copilot chat will probably suggest the following code improvement:

Copilot Proposed Change

```C#
switch (shape)
{
    case Circle circle:
        Console.WriteLine($"Processing a circle with radius {circle.Radius}");
        break;
    case Rectangle rectangle:
        Console.WriteLine($"Processing a rectangle with width {rectangle.Width} and height
{rectangle.Height}");
        break;
    case Triangle triangle:
        Console.WriteLine($"Processing a triangle with base {triangle.Base} and height
{triangle.Height}");
        break;
    default:
        Console.WriteLine("Unknown shape type");
        break;
}
```

Compare this final code to the original code at the start of the tutorial. The final code is more readable and easier to understand. It also uses the modern pattern-matching syntax to improve the control flow structure. This is a good example of how Copilot can help you write better code - even experienced programmers can learn from Copilot's suggestions.

C#: Null References

To run the following code, please create a C# console application using .NET 8, with top-level statements enabled.

1. Replace the code in the `Program.cs` file with the following code:

Program.cs

```C#
class Color
{
  int Id { get; set; }
  string Name { get; set; }
  string Hexcode { get; set; }
}
```

Review the code decorations in the Visual Studio editor. The code decorations are the colored lines and icons that appear in the Visual Studio editor. They provide information about the code, such as errors, warnings, and suggestions. The code decorations are a useful way to quickly identify issues in your code.

Are there any issues? What are they? There should be green squiggly lines under the `Name` and `Hexcode` properties. The green squiggly lines indicate that the properties could be null. Checking for possible null references in code is one of the newer features in C#. Many programmers were surprised when these null reference checks started appearing under code that had been working fine for years.

How do we learn more about this null reference checking? How do we turn it off or fix the code to properly account for null references?

2. Highlight the following line code.

Program.cs

```C#
string Name { get; set; }
```

Once the code is highlighted in the Visual Studio editor, right-click, and select `Ask Copilot` from the content menu. In the dialog that appears, ask the following question:

Copilot Chat Prompt

```TEXT
/fix why is this code complaining about a possible null reference? this code worked fine in the past, what version of C# started checking for this possible null reference? how do I disable the check?
```

Remember, Copilot is non-deterministic so the solution mentioned here may not match the solution

that Copilot provides you with. When writing this tutorial, the following solution was suggested by Copilot.

To fix the null reference warnings in your code, you can declare the Name and Hexcode properties as nullable by appending a ? to their types. This indicates that they can have a null value. Here's the updated code:

Copilot Proposed Change to Source Code

```C#
class Color
{
  int Id { get; set; }
  string? Name { get; set; }
  string? Hexcode { get; set; }
}
```

Starting from C# 8.0, the compiler introduced a nullable reference type feature that helps identify possible null reference issues at compile-time. This feature improves code safety and reduces null reference exceptions. If you want to disable the null reference check for the entire project, you can add the following line to your project file (.csproj):

Copilot Proposed Change to CSProj File

```XML
<Nullable>disable</Nullable>
```

However, it's generally recommended to embrace the null reference analysis and fix any potential null reference issues in your code. Disabling the check may lead to unexpected runtime errors.

The above response provides a code solution and it shows how to disable the null reference check for the entire project. It also explains why it's generally recommended to embrace the null reference analysis and fix any potential null reference issues in your code.

What do you think about the response? Is it helpful? What would you do next?

Copilot Chat Prompt

```text
/fix the Name and Hexcode properties will never be null. How I can tell the compiler that these properties will never be null and avoid optional chaining?
```

Copilot may provide a response similar to the following:

Copilot Proposed Change

```csharp
class Color
{
  int Id { get; set; }
  string Name { get; set; } = string.Empty;
  string Hexcode { get; set; } = string.Empty;
}
```

Would this work? Sure, but are there other options? What are they? Will Copilot tell us about them? Feel free to interact with Copilot and ask it to provide more information about the different options available to handle null references in C#.

Module 7: Fix Code with Copilot

We designed the following demonstrations to help you improve your code using GitHub Copilot. Within the module are two sections: Discuss Code and Fix Code. The Discuss Code section is for discussing code with Copilot and getting coding advice. The Fix Code section explores fixing code with Copilot. The fixes include static code errors and code patterns that lead to possible runtime errors.

C++: Convert Int to String

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Person {
  private:
    string name;
    int age;
  public:
    Person(string name, int age) {
      this->name = name;
      this->age = age;
    }
    string getName() {
      return name;
    }
    int getAge() {
      return age;
    }
    void setName(string name) {
      this->name = name;
    }
    void setAge(int age) {
      this->age = age;
    }
    string print() {
      return "Name: " + name + ", Age: " + age;
    }
};

int main() {
  Person person1("John", 30);
  cout << person1.print() << endl;
  return 0;
}
```

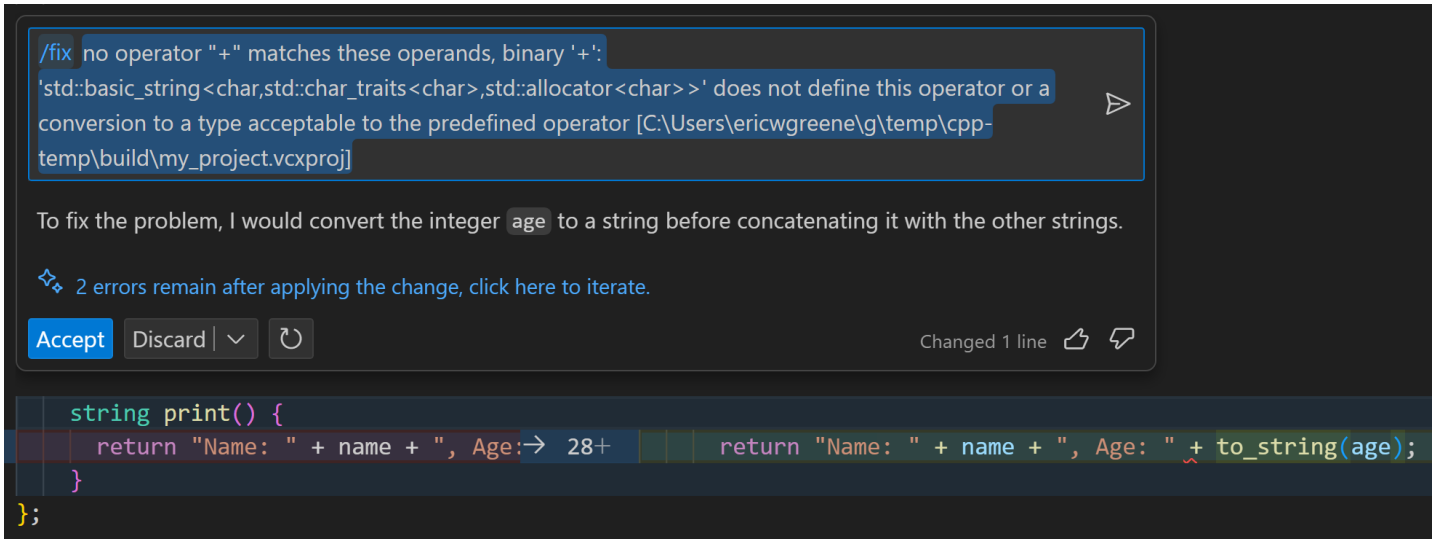
The code should fail to compile.

2. The error is located in this line of code `return "Name: " + name + ", Age: " + age;`. The variable `age` is an

integer and cannot be concatenated to a string value. To solve the problem, the integer must be converted to a string. How to do this? This is a common thing that a programmer may forget how to do if they work with many other languages, especially loosely typed languages where this conversion would happen automatically. How can Copilot help us fix it?

3. Highlight the `return "Name: " + name + ", Age: " + age;` line of code in the editor, right click, and ask Copilot to fix the code in the context menu.

Copilot should give a response similar to this:



4. Accept the change, and review the code. It should look similar to this.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Person {
  private:
    string name;
    int age;
  public:
    Person(string name, int age) {
      this->name = name;
      this->age = age;
    }
    string getName() {
      return name;
    }
    int getAge() {
      return age;
    }
    void setName(string name) {
      this->name = name;
    }
    void setAge(int age) {
      this->age = age;
    }
    string print() {
      return "Name: " + name + ", Age: " + to_string(age);
    }
};

int main() {
  Person person1("John", 30);
  cout << person1.print() << endl;
  return 0;
}
```

Compile and run the code. It should work.

C++: Detect a Memory Leak

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>

int main() {
    int *ptr = new int(5);
    std::cout << "Value: " << *ptr << std::endl;
    return 0;
}
```

Compile and run the code. It should work.

2. If you this code through a static code analysis tool, it will report a memory leak. Can you spot it? The memory leak results from failing to deallocate the memory allocated for the number 5 when assigning the `ptr` variable. Yes, the code runs but this memory leak needs to be corrected.

3. Highlight the code in the editor, right click, and ask Copilot to fix the code.

Copilot should give a response similar to this:

The provided code snippet does not seem to have any syntax errors or logical problems. It correctly allocates memory for an integer, assigns it a value of 5, and then prints that value. However, there is a memory management issue that should be addressed.

Problem 1: Memory Leak

The code allocates memory using `new` but does not deallocate it using `delete`. This can lead to a memory leak if the code is run multiple times or in a loop.

Here's how the code looks before and after the fix:

Before:

```cpp
int main() {
  int *ptr = new int(5);
  std::cout << "Value: " << *ptr << std::endl;
  return 0;
}
```

After:

```cpp
int main() {
  int *ptr = new int(5);
  std::cout << "Value: " << *ptr << std::endl;
  delete ptr;
  return 0;
}
```

In the fixed version, `delete ptr;` is added to deallocate the memory that was previously allocated with `new`.

Here's the completely fixed version of the code:

```cpp
#include <iostream>

int main() {
  int *ptr = new int(5);
  std::cout << "Value: " << *ptr << std::endl;
  delete ptr;
  return 0;
}
```

4. Using the Copilot response you received, update the code in the `main` function, and ensure it still works.

```cpp
#include <iostream>

int main() {
  int *ptr = new int(5);
  std::cout << "Value: " << *ptr << std::endl;
  delete ptr;
  return 0;
}
```

C++: Detect a Dangling Pointer

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>

int main() {
    int *ptr = new int(10);

    std::cout << "Value before delete: " << *ptr << std::endl;

    delete ptr;

    std::cout << "Value after delete: " << *ptr << std::endl;

    return 0;
}
```

Compile the code, and run it. The application will execute but the results will be strange.

2. Can you identify the dangling pointer? If you run the code through a static code analysis tool it will tell you the dangling pointer occurs when the `ptr` variable is used after the memory has been freed with the `delete` operation.

3. Highlight the code, right-click, and ask Copilot to fix the code. Apply the recommended fix and ensure the code works as expected.

4. Here is a reference solution to compare your code to.

```cpp
#include <iostream>

int main() {
  int *ptr = new int(10);

  std::cout << "Value before delete: " << *ptr << std::endl;
  delete ptr;

  return 0;
}
```

C++: Detect a Buffer Overflow

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>
#include <cstring>

int main() {
  char buffer[10];

  const char* source = "This string is too long for the buffer";

  strcpy(buffer, source);

  std::cout << "Buffer content: " << buffer << std::endl;

  return 0;
}
```

Compile the code, and run it.

2. Can you identify the buffer overflow? If you run the code through a static code analysis tool it will tell you the size of string referenced by the `source` variable is larger than the memory allocated and referenced by the `buffer` variable.

3. Highlight the code, right-click, and ask Copilot to fix the code. Apply the recommended fix and ensure the code works as expected.

4. Here is a reference solution to compare your code to.

```cpp
#include <iostream>
#include <cstring>

int main() {
  char buffer[10];

  const char* source = "This string is too long for the buffer";

  strncpy(buffer, source, sizeof(buffer) - 1);
  buffer[sizeof(buffer) - 1] = '\0'; // Ensure null-termination

  std::cout << "Buffer content: " << buffer << std::endl;

  return 0;
}
```

C++: Detect Undefined Behaviors

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>

int main() {
  int x = 10;

  x = x++ + --x;

  std::cout << "x = " << x << std::endl;

  return 0;
}
```

2. Can you identify the undefined behavior in the code? Unfortunately, even static analysis tool can

miss this potential code problem. The statement `x=x+++--x;` is undefined because the variable `x` is modified more than once between sequence points. This explanation is both obvious and possibly confusing. What is meant by sequence points? GitHub Copilot can be used to detect problems and explain concepts.

3. Select the code, and ask GitHub Copilot to fix it. The explanation of the solution will most likely mention sequence points. After reviewing the suggested solution, ask Copilot in the chat to explain the concept of sequence points.

Possible Copilot Prompt:

```text
Please explain to me what C++ sequence points are with respect to this code problem.
```

Probable Copilot Response:

```text
In C++, a sequence point defines any point in a computer program's execution at which it is
guaranteed that all side effects of previous evaluations will have been performed, and no side effects
from subsequent evaluations have yet been performed. They are used to specify the order in which
expressions are evaluated in a program.

In the context of the code problem, the expression "x = x++ + --x;" has undefined behavior because it
modifies "x" more than once between sequence points. The "++" and "--" operators each create a
sequence point, but the
order in which they are evaluated is not specified, leading to undefined behavior.

In the corrected code, we use temporary variables to hold the results of "x++" and "--x", ensuring that
"x" is not modified more than once between sequence points. This makes the behavior of the code
consistent and
predictable.
```

Yeah, that's pretty cool, right?

4. From the chat session, apply the suggested fix, and verify it generates the result you intend the code to produce. Because the original code was undefined and given to you, you get to decide how you want the calculation to be completed.

```cpp
#include <iostream>

int main() {
  int x = 10;

  int temp1 = x++;
  int temp2 = --x;
  x = temp1 + temp2;

  std::cout << "x = " << x << std::endl;

  return 0;
}
```

C++: Detect Resource Leaks

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>
#include <fstream>

void openFileAndWrite() {
  auto* file = new std::ofstream("example.txt");
  if (file->is_open()) {
    *file << "Writing some data to the file." << std::endl;
  }
}

int main() {
  openFileAndWrite();
  std::cout << "File writing function is complete, but the file is still open." << std::endl;
  return 0;
}
```

2. Do you see the two issues? First, there is a memory leak, the memory allocated for the `file` pointer is never freed. Also, because the memory is never freed, the destructor for the file stream object is never called so the file is never implicitly closed. And there is no explicit closing of the file either. The unclosed file handler is a resource leak. Some static code analysis tools will catch this, and some will not. Can GitHub Copilot catch the problem?

3. Highlight the code, and ask GitHub Copilot to review the code and suggest a fix if needed.

Copilot will probably reply with something similar to this:

Problem 1: Memory Leak

The code is creating a new `std::ofstream` object using `new`, but it never deletes this object, leading to a memory leak. Additionally, the file that is opened is never closed, which can lead to data loss or corruption.

Here's how the code looks before and after the fix:

Before:

```cpp
auto* file = new std::ofstream("example.txt");
if (file->is_open()) {
  *file << "Writing some data to the file." << std::endl;
}
```

After:

```cpp
std::ofstream file("example.txt");
if (file.is_open()) {
  file << "Writing some data to the file." << std::endl;
  file.close();
}
```

In the fixed version, we create the `std::ofstream` object on the stack, which automatically gets cleaned up when it goes out of scope. We also explicitly close the file after we're done writing to it.

Here's the completely fixed version of the code:

```cpp
#include <iostream>
#include <fstream>

void openFileAndWrite() {
  std::ofstream file("example.txt");
  if (file.is_open()) {
    file << "Writing some data to the file." << std::endl;
    file.close();
  }
}

int main() {
  openFileAndWrite();
  std::cout << "File writing function is complete, and the file is now closed." << std::endl;
  return 0;
}
```

In this fixed version, we avoid a memory leak by not using `new` to create the `std::ofstream` object, and we ensure that the file is properly closed after writing to it.

4. Fix the code as suggested, and run it to verify it still works.

```cpp
#include <iostream>
#include <fstream>

void openFileAndWrite() {
  auto* file = new std::ofstream("example.txt");
  if (file->is_open()) {
    *file << "Writing some data to the file." << std::endl;
    file->close();
  }
  delete file;
}

int main() {
  openFileAndWrite();
  std::cout << "File writing function is complete." << std::endl;
  return 0;
}
```

C++: Detect Improper Use of Pointers and References

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>

int main() {
  int* ptr = nullptr;
  int& ref = *ptr;

  int value = 10;
  ptr = &value;

  std::cout << "Value through reference: " << ref << std::endl;

  return 0;
}
```

2. Do you see the issue? A reference is created to a null pointer is created with this code `int&ref=*ptr;`. Static code analysis tools will probably pick it up. Can GitHub Copilot catch the problem?

3. Highlight the code, and ask GitHub Copilot to review the code and suggest a fix if needed.

Copilot will probably reply with something similar to this.

Problem 1: Dereferencing a Null Pointer

In the provided code, a reference is created to a null pointer. This is undefined behavior in C++ and can lead to runtime errors.

Here is the problematic code:

```cpp
int* ptr = nullptr;
int& ref = *ptr;  // Creating a reference to a null pointer
```

The fix is to ensure that the pointer points to a valid memory location before creating a reference to it.

Here is the fixed code:

```cpp
int value = 10;
int* ptr = &value;
int& ref = *ptr;  // Now the reference is to a valid memory location
```

Fixed Version of the Code

Here is the completely fixed version of the code:

```cpp
#include <iostream>

int main() {
  int value = 10;
  int* ptr = &value;
  int& ref = *ptr;  // Now the reference is to a valid memory location

  // Using the reference
  std::cout << "Value through reference: " << ref << std::endl;

  return 0;
}
```

4. Fix the code as suggested, and run it to verify it still works.

```cpp
#include <iostream>

int main() {
  int value = 10;
  int* ptr = &value;
  int& ref = *ptr;

  std::cout << "Value through reference: " << ref << std::endl;

  return 0;
}
```

C++: Detect Memory Allocation Failures

To run the following code, please create a "Hello World" style C++ project using Visual Studio or JetBrain's CLion. You may use whatever project and build tools you would like.

1. Open the C++ file with the `main` function. Replace the code in the file, with this code.

```cpp
#include <iostream>
#include <new>

int main() {
  int* largeArray = nullptr;

  largeArray = new int[1000000000000];

  delete[] largeArray;

  return 0;
}
```

2. Do you see the issue? A large array is being allocated, most likely this will fail and a `std::bad_alloc` exception will be thrown. Static code analysis tools will probably not detect this problem. Can GitHub Copilot catch the problem?

3. Highlight the code, and ask GitHub Copilot to review the code and suggest a fix if needed.

Copilot will probably reply with something similar to this:

Problem 1: Memory Allocation Size

The code is trying to allocate a very large amount of memory (1 trillion integers) which is likely to exceed the available memory, causing a `std::bad_alloc` exception to be thrown.

Before:

largeArray = new int[1000000000000];

After:

```cpp
try {
    largeArray = new int[1000000]; // Reduced size
} catch (const std::bad_alloc& e) {
    std::cerr << "Memory allocation failed: " << e.what() << '\n';
    return 1;
}
```

Fixed Version of the Code:

```cpp
#include <iostream>
#include <new>

int main() {
    int* largeArray = nullptr;

    try {
        largeArray = new int[1000000]; // Reduced size
    } catch (const std::bad_alloc& e) {
        std::cerr << "Memory allocation failed: " << e.what() << '\n';
        return 1;
    }

    delete[] largeArray;

    return 0;
}
```

This version of the code reduces the size of the memory allocation to a more reasonable amount and adds error handling for the memory allocation.

4. Fix the code as suggested, and run it to verify it still works.

```cpp
#include <iostream>
#include <new>

int main() {
  int* largeArray = nullptr;

  try {
    largeArray = new int[1000000]; // Reduced size
  } catch (const std::bad_alloc& e) {
    std::cerr << "Memory allocation failed: " << e.what() << '\n';
    return 1;
  }

  delete[] largeArray;

  return 0;
}
```

Java: Incorrect String Concatenation

To run the following code, please create a "Hello World" style Java project using Visual Studio or JetBrain's IntelliJ.

1. Open the Java file with the `main` function. Update the code in the `Main` class with this code.

```java
public class Main {
  public static void main(String[] args) {

    String message = "Sum=" + 1 + 2;

    System.out.println(message);
  }
}
```

The code should fail to run.

2. Highlight the code in the `main` function, right click, mouse over Copilot, and ask Copilot to fix the code in the context menu.

3. Copilot should offer a fix similar to this:

```java
public class Main {
  public static void main(String[] args) {

    int sum = 1 + 2;
    String message = "Sum=" + sum;

    System.out.println(message);
  }
}
```

Java: Equality Confusion

To run the following code, please create a "Hello World" style Java project using Visual Studio or JetBrain's IntelliJ.

1. Open the Java file with the `main` function. Update the code in the `Main` class with this code.

```java
public class Main {
  public static void main(String[] args) {
    String a = new String("example");
    String b = new String("example");

    if (a == b) {
      System.out.println("a and b are equal using ==");
    } else {
      System.out.println("a and b are not equal using ==");
    }
  }
}
```

The code will compile and run, but will not return the expect result of the strings being equal to each

other. Why? Ask Copilot to find out more about the problem, and how to fix it.

2. Highlight the code in the `main` function, right click, mouse over Copilot, and ask Copilot to fix the code in the context menu.

```text
/fix Why do the two string not equal each other? How do I compare the two strings to see if they are equal?
```

3. Copilot will offer an explanation and a fix similar. Read the explanation, and apply the proposed fix. The fix should look similar to this.

```java
public class Main {
    public static void main(String[] args) {
        String a = new String("example");
        String b = new String("example");

        if (a.equals(b)) {
            System.out.println("a and b are equal using .equals()");
        } else {
            System.out.println("a and b are not equal using .equals()");
        }
    }
}
```

4. Run the code. The code should run with expected result.

Python: Convert Int to String

To run the following code, please create a "Hello World" style Python project using Visual Studio or JetBrain's Pycharm.

1. Open the Python file with the `main` function. Replace the code in the file, with this code.

```
def main():
    sum = 1 + 2
    message = "Sum=" + sum
    print(message)

if __name__ == "__main__":
    main()
```

The code should fail to run.

2. Highlight the code, right click, mouse over Copilot, and ask Copilot to fix the code in the context menu.

3. Copilot should offer a fix similar to this:

```
def main():
    sum = 1 + 2
    message = "Sum=" + str(sum)
    print(message)

if __name__ == "__main__":
    main()
```

Python: Dictionary Key Error

To run the following code, please create a "Hello World" style Python project using Visual Studio or JetBrain's Pycharm.

1. Open the Python file with the `main` function. Replace the code in the file, with this code.

```java
def main():

    person = {
      "first_name": "John",
      "last_name": "Doe",
    }

    print(person["age"])


if __name__ == "__main__":
    main()
```

The code should fail to run.

2. Highlight the code, right click, mouse over Copilot, and ask Copilot to fix the code in the context menu.

3. Copilot should offer a fix similar to this:

```java
def main():

    person = {
      "first_name": "John",
      "last_name": "Doe",
      "age": 25,
    }

    print(person["age"])


if __name__ == "__main__":
    main()
```

4. Accept the fix and run the code. The code should run without errors.

5. Undo the fix, and ask Copilot to fix the code again. But the add the following text following the `/fix` command:

```
/fix improve the code so it works even if the age key is not present in the dictionary
```

6. Copilot should offer a fix similar to this:

```
To improve the code so it works even if the "age" key is not present in the dictionary, I would use the get() method to retrieve the value of the "age" key, and provide a default value of 0 if the key is not found.
```

```java
def main():

    person = {
      "first_name": "John",
      "last_name": "Doe",
    }

    age = person.get("age", 0)
    print(age)


if __name__ == "__main__":
    main()
```

7. Accept the fix and run the code. The code should run without errors.

C#: Index Out of Range

To run the following code, please create a C# console application using .NET 8, with top-level statements enabled.

1. Replace the code in the `Program.cs` file with the following code:

Program.cs

```C#
decimal[] prices = { 9.99m, 10.00m, 20.50m, 30.00m, 40.00m };
decimal totalPrice = CalculateTotalPrice(prices);
Console.WriteLine($"Total price: {totalPrice:C}");

static decimal CalculateTotalPrice(decimal[] prices)
{
  decimal total = 0;
  for (int i = 0; i <= prices.Length; i++)
  {
    total += prices[i];
  }
  return total;
}
```

2. Highlight the code in the function `CalculateTotalPrice`, right-click, and select `Ask Copilot`.

Program.cs

```C#
static decimal CalculateTotalPrice(decimal[] prices)
{
  decimal total = 0;
  for (int i = 0; i <= prices.Length; i++)
  {
    total += prices[i];
  }
  return total;
}
```

In the Copilot dialog, enter the following prompt:

Copilot Chat Prompt

```TEXT
/fix
```

Copilot Proposed Change

```C#
static decimal CalculateTotalPrice(decimal[] prices)
{
  decimal total = 0;
  for (int i = 0; i < prices.Length; i++)
  {
    total += prices[i];
  }
  return total;
}
```

The less-than or equal to operator in the for loop condition should be replaced with only a less-than operator. Copilot can detect some runtime errors, it is not limited to just syntax errors.

C#: Async without Await

To run the following code, please create a C# console application using .NET 8, with top-level statements enabled.

1. Replace the code in the `Program.cs` file with the following code:

Program.cs

C#

```
IColorsData colorsData = new ColorsInMemoryData();
var colors = await colorsData.AllColors();
foreach (var color in colors)
{
  Console.WriteLine($"Color: {color.Name}, HexCode: {color.HexCode}");
}

class ColorsInMemoryData: IColorsData
{
    private List<Color> _colors = new List<Color>
    {
    new Color {Id = 1, Name = "Red", HexCode = "#FF0000"},
    new Color {Id = 2, Name = "Green", HexCode = "#00FF00"},
    new Color {Id = 3, Name = "Blue", HexCode = "#0000FF"}
  };

  public async Task<IEnumerable<Color>> AllColors()
  {
    return _colors;
  }
}

class ColorsSqlData: IColorsData
{
  private readonly string _connectionString;
  public ColorsSqlData(string connectionString)
  {
    _connectionString = connectionString;
  }

  public Task<IEnumerable<Color>> AllColors()
  {
    throw new NotImplementedException();
  }
}


record Color
{
  public int Id { get; init;}
  public required string Name { get; init; }
  public required string HexCode { get; init; }
```

```
}

interface IColorsData
{
  Task<IEnumerable<Color>> AllColors();
}
```

The code will compile and run, but the editor complains that `await` needs to be called in the `async` marked method in the `ColorsInMemoryData` class. The `AllColors` method is marked as `async` but does not contain an `await` statement.

Program.cs

```C#
public async Task<IEnumerable<Color>> AllColors()
{
  return _colors;
}
```

Ask Copilot the following question:

Copilot Chat Prompt

```TEXT
/fix how do I fake an asynchronous operation in the AllColors method without using async/await?
```

3. Copilot should make a suggestion similar to this.

Copilot Proposed Change

```C#
public Task<IEnumerable<Color>> AllColors()
{
  return Task.FromResult<IEnumerable<Color>>(_colors);
}
```

Accept the solution and run the code. The code will compile and run successfully. Also, there should be no more warnings in the editor.

C#: Blocking Call

To run the following code, please create a C# console application using .NET 8, with top-level statements enabled.

1. Replace the code in the `Program.cs` file with the following code:

Program.cs

```C#
ProcessData();

void ProcessData()
{
  var result = FetchDataAsync().Result;
  Console.WriteLine(result);
}

async Task<string> FetchDataAsync()
{
  // Simulating an asynchronous operation
  await Task.Delay(1000);
  return "Data";
}
```

Does this code have any potential problems? If so, what are they?

Program.cs

```C#
void ProcessData()
{
  var result = FetchDataAsync().Result;
  Console.WriteLine(result);
}
```

Ask Copilot the following question:

Copilot Chat Prompt

```text
/fix does this code have any potential problems?
```

3. Copilot should respond with some advice similar to this.

Copilot Response

The code snippet you provided has a potential problem known as "deadlock". The use of the .Result property on the Task returned by FetchDataAsync() can cause the calling thread to block until the asynchronous operation completes. However, in this case, the calling thread is the main thread of the application, which is blocked waiting for the asynchronous operation to complete. This can lead to a deadlock situation if the asynchronous operation requires the main thread to continue executing.

To fix this problem, you should use the await keyword instead of blocking on the Result property. Here's the updated code:

Copilot Proposed Change

```C#
async Task ProcessData()
{
  var result = await FetchDataAsync();
  Console.WriteLine(result);
}
```

4. The proposed solution is to use the `await` keyword instead of blocking on the `Result` property. This will allow the main thread to continue executing while the asynchronous operation completes. Update the code and the rest of the program to use this approach.

Module 8: Workshops

Following the training session, we encourage you to explore the workshops below to deepen your skills with GitHub Copilot. These language-specific workshops will help you understand how to use Copilot in a real-world scenario.

Build a Calculator with C++

Setup

This tutorial requires Microsoft Windows with Visual Studio Community Edition and MinGW installed. MinGW can be downloaded from Winlibs (https://winlibs.com/). It is recommended to download the latest UCRT version for your platform, x86 or x64. Finally, extract the archive, and configure your `PATH` to include the `bin` folder of the extracted archive.

Steps for Part 1

1. Create a new folder named `calculator-cpp`. Open the folder with VS Code.

2. Ask Copilot how to create a new C++ main file.

Copilot Chat Prompt:

```text
I am working on a new C++ project. Please generate a main file that prints hello world to the console.
```

Probable Partial Response:

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

3. Ask Copilot how to build and run the program.

Copilot Chat Prompt:

```text
```

How can I compile and run the C++ program you provided? The name of the executable file must be calculator.exe.

Probable Partial Response:

```bash
g++ main.cpp -o calculator.exe
```

- Open a terminal, and run the following command.

```bash
g++ main.cpp -o calculator.exe
```

Then, run the program.

```bash
calculator.exe
```

4. Ask Copilot to generate a CMake configuration.

Copilot Chat Prompt:

```text
Generate a new CMake configuration file for my project named CalculatorCpp. Use cmake version 3.27 and C++ version 23.
```

Probable Partial Response:

```cmake
cmake_minimum_required(VERSION 3.27)  # CMake 3.27 or later is required

# set the project name
project(CalculatorCpp)

# specify the C++ standard
set(CMAKE_CXX_STANDARD 23)

# add the executable
add_executable(CalculatorCpp main.cpp)
```

5. In the project's root folder, create a new file named `CMakeLists.txt` and add the following content to the file.

```cmake
cmake_minimum_required(VERSION 3.27)  # CMake 3.27 or later is required

# set the project name
project(CalculatorCpp)

# specify the C++ standard
set(CMAKE_CXX_STANDARD 23)

# add the executable
add_executable(CalculatorCpp main.cpp)
```

6. Ask Copilot how to build the project with cmake.

Copilot Chat Prompt:

```test
How do I build the project with CMake?
```

Probable Partial Response:

```bash
mkdir build
cd build

cmake ..

make
```

7. Run the following commands. The `make` command will fail. Instead, run `mingw2-make`. It will fail too, because there is no `Makefile.`

```bash
mkdir build
cd build

cmake ..

make

mingw2-make
```

8. Ask GitHub Copilot how to build on Windows when there is no `Makefile` but there is a Visual Studio Solution file.

Copilot Chat Prompt:

```text
When running make, I receive an error saying there is no make target, but there is a Visual Studio Solution file. How do I use the CMmake command to execute a build the Visual Studio Solution?
```

Probable Partial Response:

```bash
cmake --build .
```

9. Run the following CMake build command, then run the executable.

```bash
cmake --build .

Debug\CalculatorCpp.exe
```

10. Create a new local git repository.

```bash
git init
```

11. Ask GitHub Copilot how to create a `.gitignore` file for a C++ project.

Copilot Chat Prompt:

```text
Generate a Git ignore file for my C++ and CMake project.
```

Probable Partial Response:

```text
The contents of a Git ignore file.
```

12. Create a new `.gitignore` file, and add the contents suggested by the previous response.

13. Add the untracked file, create a commit, and push to a remote.

```bash
git add -A

git commit -m "initial commit"

git remote add origin <SOME REMOTE>

git push
```

14. Open the file `main.cpp` in the editor. Then, open the Copilot chat window.

15. Ask Copilot to how to initialize a result variable to 0 and display the result on the screen.

Copilot Chat Prompt:

```text
How do I initialize a result variable to 0 and display the result on the screen?
```

Probable Partial Response:

```cpp
#include <iostream>

int main() {
    int result = 0;
    std::cout << "The result is: " << result << std::endl;
    return 0;
}
```

16. Update the `main` function with the following code:

```cpp
int main() {

  int result = 0;

  std::cout << "The result is: " << result << std::endl;
  return 0;
}
```

Run the project, and verify it works.

Ask Copilot to how to read numeric input from the user and add the entered number to the result.

Copilot Chat Prompt:

TEXT

How does the program accept user input from the console? I would like to have the user enter a number, then add the number to the result.

Probable Partial Response:

CPP

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;

    std::cout << "Enter a number: ";
    std::cin >> input;

    result += input;

    std::cout << "The result is: " << result << std::endl;
    return 0;
}
```

18. Update the `main` function to use the following code:

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;

    std::cout << "Enter a number: ";
    std::cin >> input;

    result += input;

    std::cout << "The result is: " << result << std::endl;
    return 0;
}
```

Compile and run the code. It should work as expected.

19. Ask Copilot to add a loop so the user can enter multiple numbers and continue to add to the result.

Copilot Chat Prompt:

```text
How do I implement a loop so that users can add many numbers to the result?
```

Probable Partial Response:

To implement a loop that allows users to add multiple numbers to the result, you can use a `while` loop. Here's the step-by-step plan:

- Initialize a `char` variable to store the user's choice to continue or not.
- Start a `do-while` loop. This type of loop will execute the code block at least once before checking the condition.
- Inside the loop, prompt the user to enter a number.
- Read the user's input and add it to the result.
- Display the updated result.

- Ask the user if they want to enter another number.
- Read the user's choice.
- The loop will continue as long as the user enters 'y' or 'Y'.

Here's the code:

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;
    char choice;

    do {
        std::cout << "Enter a number: ";
        std::cin >> input;

        result += input;

        std::cout << "The result is: " << result << std::endl;

        std::cout << "Do you want to enter another number? (y/n): ";
        std::cin >> choice;
    } while(choice == 'y' || choice == 'Y');

    return 0;
}
```

This code will keep asking the user to enter a number and add it to the result until the user decides to stop by entering anything other than 'y' or 'Y'.

20. Using the last Copilot response, update your code, and run the program. Does it work? If it works, then great! If not, continue to step 8 and compare your code to the reference code.

21. Compare your code updates in the previous step to the code below. If your code does not work, please update accordingly.

```cpp
#include <iostream>

int main() {

  int result = 0;
  int input;
  char choice;

  do {

    std::cout << "Enter a number: ";
    std::cin >> input;

    result += input;

    std::cout << "The result is: " << result << std::endl;

    std::cout << "Do you want to enter another number? (y/n): ";
    std::cin >> choice;

  } while (choice == 'y' || choice == 'Y');

  return 0;
}
```

22. Ask the Copilot chat how to support multiple math operations.

Copilot Chat Prompt:

```text
How do I update the program to support add, subtract, multiply, and divide?
The user must choose a math operation on each iteration of the command loop.
```

Probable Partial Response:

CPP

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;
    char choice;
    char operation;

    do {
        std::cout << "Enter a number: ";
        std::cin >> input;

        std::cout << "Choose an operation (+, -, *, /): ";
        std::cin >> operation;

        switch (operation) {
            case '+':
                result += input;
                break;
            case '-':
                result -= input;
                break;
            case '*':
                result *= input;
                break;
            case '/':
                if (input != 0) {
                    result /= input;
                } else {
                    std::cout << "Error: Division by zero is not allowed.\n";
                }
                break;
            default:
                std::cout << "Error: Invalid operation. Please choose +, -, *, or /.\n";
                continue; // Skip the rest of the loop for this iteration
        }

        std::cout << "The result is: " << result << std::endl;

        std::cout << "Do you want to enter another number? (y/n): ";
        std::cin >> choice;
    } while (choice == 'y' || choice == 'Y');
```

```

    return 0;
}
```

Using the response from Copilot, please update your code.

23. Compare you code to the reference code below. Make any needed adjustments. Your code does not have to match this code exactly. Simply use the code to ensure similar functionality and to resolve any bugs you may have.

CPP

```cpp
#include <iostream>

int main() {

 int result = 0;
 int input;
 char choice;
 char operation;

 do {

  std::cout << "Enter a number: ";
  std::cin >> input;

  std::cout << "Choose an operation (+, -, *, /): ";
  std::cin >> operation;

  switch (operation) {
   case '+':
    result += input;
    break;
   case '-':
    result -= input;
    break;
   case '*':
    result *= input;
    break;
   case '/':
    // -- used the comment below to encourage Copilot suggestion --
    // Check for division by zero
    if (input == 0) {
     std::cout << "Division by zero is not allowed." << std::endl;
    } else {
     result /= input;
    }
    break;
   default:
    // -- used the comment below to encourage Copilot suggestion --
    // show an invalid operation message that includes a list of valid
    // operations
    std::cout << "Error: Invalid operation. Please choose +, -, *, /." <<
    std::endl;
```

```
        continue;
    }

    std::cout << "The result is: " << result << std::endl;

    std::cout << "Do you want to enter another number? (y/n): ";
    std::cin >> choice;

  } while (choice == 'y' || choice == 'Y');

  return 0;
}
```

24. For part, one final improvement is needed. The user input for the math operation command and the exit loop command need to be combined. Copilot can help us do this. Ask Copilot to refactor our logic to reduce the code to single user input for math operations and loop exit. For this step, try writing your own Copilot prompt and follow the suggestion offer by Copilot. A suggested prompt and solution is offered in the next step.

The Copilot prompt and code below are the solution for the previous step.

Copilot Chat Prompt:

```text
Combine the user input for the math operation command and the exit loop command into a single input and update the appropriate control flow logic. The user input to capture the command/exit should be a different user input than capturing the number.
```

Probable Partial Response:

To achieve this, you can modify the program to first ask for the operation and then ask for the number. If the user enters 'q' or 'Q' as the operation, the program will quit. Otherwise, it will ask for a number and perform the chosen operation on the result and the user's input. Here's the updated code:

CPP

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;
    char operation;

    while (true) {
        std::cout << "Enter an operation (+, -, *, /) or 'q' to quit: ";
        std::cin >> operation;

        if (operation == 'q' || operation == 'Q') {
            break;
        }

        std::cout << "Enter a number: ";
        std::cin >> input;

        switch (operation) {
            case '+':
                result += input;
                break;
            case '-':
                result -= input;
                break;
            case '*':
                result *= input;
                break;
            case '/':
                if (input != 0) {
                    result /= input;
                } else {
                    std::cout << "Error: Division by zero is not allowed.\n";
                }
                break;
            default:
                std::cout << "Error: Invalid operation. Please choose +, -, *, or /.\n";
                continue; // Skip the rest of the loop for this iteration
        }

        std::cout << "The result is: " << result << std::endl;
    }
```

```

    return 0;
}
```

This code will first prompt the user to enter an operation. If the user enters 'q' or 'Q', the program will quit. Otherwise, it will prompt the user to enter a number and perform the chosen operation on the result and the user's input. If the user chooses division and enters zero as the number, the program will print an error message and ask for another operation and number.

Workshop Reference Solution

The following code if the final reference solution for the tutorial. Please compare your code to ensure similar functionality and that your code works as expected.

CPP

```cpp
#include <iostream>

int main() {
    int result = 0;
    int input;
    char operation;

    while (true) {
        std::cout << "Enter an operation (+, -, *, /) or 'q' to quit: ";
        std::cin >> operation;

        if (operation == 'q' || operation == 'Q') {
            break;
        }

        std::cout << "Enter a number: ";
        std::cin >> input;

        switch (operation) {
            case '+':
                result += input;
                break;
            case '-':
                result -= input;
                break;
            case '*':
                result *= input;
                break;
            case '/':
                if (input != 0) {
                    result /= input;
                } else {
                    std::cout << "Error: Division by zero is not allowed.\n";
                }
                break;
            default:
                std::cout << "Error: Invalid operation. Please choose +, -, *, or /.\n";
                continue; // Skip the rest of the loop for this iteration
        }

        std::cout << "The result is: " << result << std::endl;
    }
```

```

    return 0;
}
```

C++: Write a Unit Test

Setup

This tutorial requires Microsoft Windows with Visual Studio 2022 Community Edition. Also, Visual Studio Code will need Microsoft's C++ and CMake extensions.

Steps

1. Copy the folder `begin-code`. Open the copied folder with Visual Studio Code.

2. Open the file `utils.cpp`, review the `add` function. A unit test is needed for the `add` function. How would you write this? A popular unit testing library is Google Test (https://google.github.io/googletest/). Do you know how to wire up the test framework and write the test off the top of your head? Do you work on projects where the testing framework is already configured? How would you setup unit testing for this project?

3. Let's start with learn how to configure unit testing for the project.

Copilot Chat Prompt:

```text
How do I add Google Test to an existing C++ project that is using CMake as the build tool? I would like CMake to fetch and install the release zip file for Google Test when building. Also, how do I configure a build target for running the tests.
```

Probable Copilot Response*

To add Google Test to a C++ project using CMake, have CMake install the latest release zip file for Google Test during the build process, and configure a build target for running the tests, you can follow these steps:

- In your `CMakeLists.txt`, include the `FetchContent` module:

```cmake
include(FetchContent)
```

- Use `FetchContent_Declare` to specify the Google Test repository:

```cmake
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/release-1.11.0.zip
)
```

- Use `FetchContent_MakeAvailable` to download and set up Google Test:

```cmake
FetchContent_MakeAvailable(googletest)
```

- Create a new executable for your tests:

```cmake
add_executable(tests your_test_file.cpp)
```

- Link Google Test to your test executable:

```cmake
target_link_libraries(tests gtest_main)
```

- Add a test command to enable testing:

```cmake
enable_testing()
```

- Add your test executable as a test:

```cmake
add_test(NAME example_test COMMAND tests)
```

Remember to replace `your_test_file.cpp` with your actual test file name. Now, when you build your project, CMake will automatically download and set up Google Test. You can run the tests by using the `ctest` command in the command prompt.

Please note that you need to have CMake and a C++ compiler installed on your machine. If you're on Windows, you can use the Visual Studio compiler, or install MinGW or Cygwin.

4. Update the `CMakeLists.txt` file to contain this code. This code will be similar to the suggested code with source file names update for the specifics of this project. Sometimes, Copilot gets the solution perfectly sometimes it gets the solution most of the way there.

```cmake
cmake_minimum_required(VERSION 3.27)  # CMake 3.27 or later is required

# set the project name
project(CalculatorCpp)

include(FetchContent)
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
)
FetchContent_MakeAvailable(googletest)


# specify the C++ standard
set(CMAKE_CXX_STANDARD 23)

# add the executable
add_executable(CalculatorCpp
        main.cpp
        utils.cpp
        utils.h)

add_executable(tests test_utils.cpp utils.cpp utils.h)
target_link_libraries(tests gtest_main)
enable_testing()

add_test(NAME tests COMMAND tests)
```

5. Next, a new unit test is needed for the `add` function. Highlight the `add` function in the `utils.cpp` file. Right click on the selection and have Copilot "Generate Tests". The code will look similar to this. Copilot may offer to create a new unit test file named something similar to `test_utils.cpp`. If it offers, allow it to create the file, and make sure it is in the same folder as the `utils.cpp` file. If it does not offer, then create a file in the same folder as the `utils.cpp` file, and copy the suggested code into it.

CPP

```cpp
#include <gtest/gtest.h>

#include "utils.h"

TEST(AddFunction, ReturnsSumOfTwoPositiveNumbers) {
  EXPECT_EQ(add(1, 2), 3);
}

TEST(AddFunction, ReturnsSumOfTwoNegativeNumbers) {
  EXPECT_EQ(add(-1, -2), -3);
}

TEST(AddFunction, ReturnsZeroWhenAddingPositiveAndNegativeOfSameValue) {
  EXPECT_EQ(add(5, -5), 0);
}

TEST(AddFunction, ReturnsSumOfZeroAndNumber) {
  EXPECT_EQ(add(0, 5), 5);
}

TEST(AddFunction, ReturnsSumOfNumberAndZero) {
  EXPECT_EQ(add(5, 0), 5);
}

TEST(AddFunction, ReturnsSumOfTwoLargeNumbers) {
  EXPECT_EQ(add(1000000, 2000000), 3000000);
}

TEST(AddFunction, ReturnsSumOfTwoNegativeLargeNumbers) {
  EXPECT_EQ(add(-1000000, -2000000), -3000000);
}

TEST(AddFunction, ReturnsSumOfLargeNumberAndZero) {
  EXPECT_EQ(add(5000000, 0), 5000000);
}

TEST(AddFunction, ReturnsSumOfZeroAndLargeNumber) {
  EXPECT_EQ(add(0, 5000000), 5000000);
}
```

6. Using the CMake extension, run the tests. The test should pass. Update one of the tests to make it fail.

Build a Calculator with Java

Setup

To complete the following workshop, please create a Java console application.

Part 1: Basic Calculator Steps

Note: For Part 1, sample Copilot chat prompts and suggested code are provided. For Part 2 and Part 3, you will need to create your own chat prompts and use the suggestions from Copilot to update the code.

1. Create a new project and open the project with IntelliJ IDEA.
2. Open Copilot Chat, and explain to Copilot that you want to build a console-based calculator in Java. Ask Copilot to help you write the code. Start by asking Copilot to write a command loop that will keep asking the user for input until the user types "exit". For any command other than "exit", print the command in the console. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```text
I am working on new Java console-based Calculator program. I will need some help writing the code. Can you please create a basic command loop that will ask users for a new command until the user types "exit". When the user types "exit" the program should end. For any other comment, please display the command in the console before asking for the next command.
```

Copilot Suggested Code

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();
            if (!command.equalsIgnoreCase("exit")) {
                System.out.println("You entered: " + command);
            }
        } while (!command.equalsIgnoreCase("exit"));

        scanner.close();
        System.out.println("Program ended.");
    }
}
```

3. Next, ask Copilot to track the current calculator state in a variable named "result". Ask Copilot to initialize the variable to 0. Then, ask Copilot to add a new command to add a number to the result. When the user types "add 5", for example, the result should be updated to 5. If the user types "add 3", the result should be updated to 8. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```text
Track current state of the calculator with a result variable initialized to 0. The result should be a floating point number. Also, please add one command for adding a number to the result. The command should display the new result after performing the add operation. The format of the command should be similar to this "add 3" which would then add 3 to the result and display the new result.
```

Copilot Suggested Code

JAVA

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        double result = 0.0;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase("exit")) {
                String[] parts = command.split(" ");
                if (parts[0].equalsIgnoreCase("add") && parts.length > 1) {
                    try {
                        double numberToAdd = Double.parseDouble(parts[1]);
                        result += numberToAdd;
                        System.out.println("New result: " + result);
                    } catch (NumberFormatException e) {
                        System.out.println("Invalid number format. Please enter a valid number to add.");
                    }
                } else {
                    System.out.println("You entered: " + command);
                }
            }
        } while (!command.equalsIgnoreCase("exit"));

        scanner.close();
        System.out.println("Program ended.");
    }
}
```

4. Verify the code created so far works. Add a new command named "result" to display the current result whenever the user asks. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

TEXT

Add a command to display the current result. Also, if the user types an unrecognized command, please display an error message and allow the user to type another command.

Copilot Suggested Code

JAVA

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        double result = 0.0;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase("exit")) {
                String[] parts = command.split(" ");
                if (parts[0].equalsIgnoreCase("add") && parts.length > 1) {
                    try {
                        double numberToAdd = Double.parseDouble(parts[1]);
                        result += numberToAdd;
                        System.out.println("New result: " + result);
                    } catch (NumberFormatException e) {
                        System.out.println("Invalid number format. Please enter a valid number to add.");
                    }
                } else if (parts[0].equalsIgnoreCase("result")) {
                    System.out.println("Current result: " + result);
                } else {
                    System.out.println("Unrecognized command. Please enter a valid command.");
                }
            }
        } while (!command.equalsIgnoreCase("exit"));

        scanner.close();
        System.out.println("Program ended.");
    }
}
```

Sample Copilot Chat Prompt

TEXT

Please refactor the code to make it more readable.

Copilot Suggested Code

JAVA

```java
import java.util.Scanner;

public class Main {
    private static final String EXIT_COMMAND = "exit";
    private static final String ADD_COMMAND = "add";
    private static final String RESULT_COMMAND = "result";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        double result = 0.0;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase(EXIT_COMMAND)) {
                result = processCommand(command, result);
            }
        } while (!command.equalsIgnoreCase(EXIT_COMMAND));

        scanner.close();
        System.out.println("Program ended.");
    }

    // Process the user's command and return the updated result
    private static double processCommand(String command, double currentResult) {
        String[] parts = command.split(" ");
        if (parts[0].equalsIgnoreCase(ADD_COMMAND) && parts.length > 1) {
            currentResult = processAddCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(RESULT_COMMAND)) {
            displayResult(currentResult);
        } else {
            System.out.println("Unrecognized command. Please enter a valid command.");
        }
        return currentResult;
    }

    // Process the add command and return the updated result
    private static double processAddCommand(String numberString, double currentResult) {
        try {
            double numberToAdd = Double.parseDouble(numberString);
```

```
            currentResult += numberToAdd;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to add.");
        }
        return currentResult;
    }

    // Display the current result
    private static void displayResult(double result) {
        System.out.println("Current result: " + result);
    }
}
```

6. Next, ask Copilot to add new commands for subtraction, multiplication, and division. Ensure there is the error handling for division by zero. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```
TEXT
Please add commands for subtraction, multiplication, and division. Also, ensure there is error handling for division by zero.
```

Copilot Suggested Code

JAVA

```java
import java.util.Scanner;

public class Main {
    private static final String EXIT_COMMAND = "exit";
    private static final String ADD_COMMAND = "add";
    private static final String SUBTRACT_COMMAND = "subtract";
    private static final String MULTIPLY_COMMAND = "multiply";
    private static final String DIVIDE_COMMAND = "divide";
    private static final String RESULT_COMMAND = "result";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        double result = 0.0;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase(EXIT_COMMAND)) {
                result = processCommand(command, result);
            }
        } while (!command.equalsIgnoreCase(EXIT_COMMAND));

        scanner.close();
        System.out.println("Program ended.");
    }

    // Process the user's command and return the updated result
    private static double processCommand(String command, double currentResult) {
        String[] parts = command.split(" ");
        if (parts[0].equalsIgnoreCase(ADD_COMMAND) && parts.length > 1) {
            currentResult = processAddCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(SUBTRACT_COMMAND) && parts.length > 1) {
            currentResult = processSubtractCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(MULTIPLY_COMMAND) && parts.length > 1) {
            currentResult = processMultiplyCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(DIVIDE_COMMAND) && parts.length > 1) {
            currentResult = processDivideCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(RESULT_COMMAND)) {
            displayResult(currentResult);
        } else {
```

```
            System.out.println("Unrecognized command. Please enter a valid command.");
        }
        return currentResult;
    }

    // Process the add command and return the updated result
    private static double processAddCommand(String numberString, double currentResult) {
        try {
            double numberToAdd = Double.parseDouble(numberString);
            currentResult += numberToAdd;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to add.");
        }
        return currentResult;
    }

    // Process the subtract command and return the updated result
    private static double processSubtractCommand(String numberString, double
currentResult) {
        try {
            double numberToSubtract = Double.parseDouble(numberString);
            currentResult -= numberToSubtract;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to
subtract.");
        }
        return currentResult;
    }

    // Process the multiply command and return the updated result
    private static double processMultiplyCommand(String numberString, double
currentResult) {
        try {
            double numberToMultiply = Double.parseDouble(numberString);
            currentResult *= numberToMultiply;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to multiply.");
        }
        return currentResult;
    }

```

```
    // Process the divide command and return the updated result
    private static double processDivideCommand(String numberString, double
currentResult) {
        try {
            double numberToDivide = Double.parseDouble(numberString);
            if (numberToDivide == 0) {
                System.out.println("Error: Division by zero is not allowed.");
            } else {
                currentResult /= numberToDivide;
                System.out.println("New result: " + currentResult);
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to divide.");
        }
        return currentResult;
    }

    // Display the current result
    private static void displayResult(double result) {
        System.out.println("Current result: " + result);
    }
}
```

7. Finally, ask Copilot to add a command for clearing the result. When the user types "clear", the result should be reset to 0. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```TEXT
Please add a command for clearing the result. When the user types "clear", the result should be reset to 0.
```

Copilot Suggested Code

JAVA

```java
import java.util.Scanner;

public class Main {
    private static final String EXIT_COMMAND = "exit";
    private static final String ADD_COMMAND = "add";
    private static final String SUBTRACT_COMMAND = "subtract";
    private static final String MULTIPLY_COMMAND = "multiply";
    private static final String DIVIDE_COMMAND = "divide";
    private static final String RESULT_COMMAND = "result";
    private static final String CLEAR_COMMAND = "clear";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        double result = 0.0;

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase(EXIT_COMMAND)) {
                result = processCommand(command, result);
            }
        } while (!command.equalsIgnoreCase(EXIT_COMMAND));

        scanner.close();
        System.out.println("Program ended.");
    }

    // Process the user's command and return the updated result
    private static double processCommand(String command, double currentResult) {
        String[] parts = command.split(" ");
        if (parts[0].equalsIgnoreCase(ADD_COMMAND) && parts.length > 1) {
            currentResult = processAddCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(SUBTRACT_COMMAND) && parts.length > 1) {
            currentResult = processSubtractCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(MULTIPLY_COMMAND) && parts.length > 1) {
            currentResult = processMultiplyCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(DIVIDE_COMMAND) && parts.length > 1) {
            currentResult = processDivideCommand(parts[1], currentResult);
        } else if (parts[0].equalsIgnoreCase(RESULT_COMMAND)) {
            displayResult(currentResult);
```

```
        } else if (parts[0].equalsIgnoreCase(CLEAR_COMMAND)) {
            currentResult = 0.0;
            System.out.println("Result cleared.");
        } else {
            System.out.println("Unrecognized command. Please enter a valid command.");
        }
        return currentResult;
    }

    // Process the add command and return the updated result
    private static double processAddCommand(String numberString, double currentResult) {
        try {
            double numberToAdd = Double.parseDouble(numberString);
            currentResult += numberToAdd;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to add.");
        }
        return currentResult;
    }

    // Process the subtract command and return the updated result
    private static double processSubtractCommand(String numberString, double
currentResult) {
        try {
            double numberToSubtract = Double.parseDouble(numberString);
            currentResult -= numberToSubtract;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to
subtract.");
        }
        return currentResult;
    }

    // Process the multiply command and return the updated result
    private static double processMultiplyCommand(String numberString, double
currentResult) {
        try {
            double numberToMultiply = Double.parseDouble(numberString);
            currentResult *= numberToMultiply;
            System.out.println("New result: " + currentResult);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to multiply.");
```

```
            System.out.println("Invalid number format. Please enter a valid number to multiply.");
        }
        return currentResult;
    }

    // Process the divide command and return the updated result
    private static double processDivideCommand(String numberString, double
currentResult) {
        try {
            double numberToDivide = Double.parseDouble(numberString);
            if (numberToDivide == 0) {
                System.out.println("Error: Division by zero is not allowed.");
            } else {
                currentResult /= numberToDivide;
                System.out.println("New result: " + currentResult);
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to divide.");
        }
        return currentResult;
    }

    // Display the current result
    private static void displayResult(double result) {
        System.out.println("Current result: " + result);
    }
}
```

If Copilot places the `Calculator` instantiation and `Run` method call at the bottom of the file, move it to the top of the `Program.cs` file.

Part 2: Calculator History Steps

Note: For Part 2, the steps are listed, but no sample prompts or code are provided. You will need to create your own chat prompts and use the suggestions from Copilot to update the code.

1. Ask Copilot to track the history of all commands entered by the user. When the user types "history", the console should display all the commands entered so far. Using the suggestions from Copilot, update the code.
2. Add a command to remove a history entry from the list of commands. When the user types "remove 3", for example, the third command should be removed from the history. Using the suggestions from Copilot, update the code.

3. Ask Copilot how to modify the code to display the result based upon the `history` list not the `result` variable. Remove the `result` variable from the code. Using the suggestions from Copilot, update the code.

Note: The solution to part 2 is located at the of this workshop.

Part 3: History File Operations Steps

Note: For Part 3, a user story describes your task. Complete the user story using your AI pair-programming skills. Unlike Part 1 and 2, there is no solution provided for Part 3. Good luck!

User Story: Persist and Reload Calculator Command History

As a user of the calculator application, I want the ability to persist the history of my calculator commands to a file and reload those commands from the file back into the calculator. This will allow me to save my calculation history for future reference and easily restore my previous calculations when needed.

Acceptance Criteria:
1. The calculator application should provide an option to save the command history to a file.
  - The user should be able to specify the location and name of the file to save the command history.
  - The command history should be saved in JSON.
  - The user should receive confirmation that the command history has been successfully saved to the file.

2. The calculator application should provide an option to load a previously saved command history file.
  - The user should be able to select the file containing the command history to be loaded.
  - The application should validate the file format and ensure that it contains valid command history data.
  - If the file is valid, the command history should be loaded into the calculator, replacing any existing command history.
  - The user should receive confirmation that the command history has been successfully loaded from the file.

3. The loaded command history should be accessible and usable within the calculator.
  - The user should be able to view the loaded command history, including all previous commands and their results.
  - The user should be able to navigate through the loaded command history, similar to the existing functionality for navigating the command history within the calculator.
  - The user should be able to execute commands from the loaded command history, as if they were entered manually.

4. Do not worry about implementing error handling for reading and writing to the file.

Solution for Part 2

Copilot Chat Prompt #1

TEXT

Enhance the application to remember only the history of math commands (add, subtract, multiply, divide) typed by the user. Each command in the history should be an object with a field for the command name and the number entered by the user. Also, include a new command named "history" that will display the history of math operations. The clear command should clear the history.

Copilot Chat Prompt #2

```TEXT
Enhance the code to remove an entry from the history using a new command named "remove". The structure for the command is "remove 3" where 3 is the index of the command to remove. If there is no command at that index, please display an error message and allow the user to enter another command.
```

Copilot Chat Prompt #3

```TEXT
Remove the result variable, and calculate the result using the history of math commands.
```

Final Code Solution to Part 2, Main.java

JAVA

```java
package com.training4programmers;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Main {
    private static final String EXIT_COMMAND = "exit";
    private static final String ADD_COMMAND = "add";
    private static final String SUBTRACT_COMMAND = "subtract";
    private static final String MULTIPLY_COMMAND = "multiply";
    private static final String DIVIDE_COMMAND = "divide";
    private static final String RESULT_COMMAND = "result";
    private static final String CLEAR_COMMAND = "clear";
    private static final String HISTORY_COMMAND = "history";
    private static final String REMOVE_COMMAND = "remove";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String command;
        List<Command> history = new ArrayList<>();

        do {
            System.out.println("Enter a command:");
            command = scanner.nextLine();

            if (!command.equalsIgnoreCase(EXIT_COMMAND)) {
                processCommand(command, history);
            }
        } while (!command.equalsIgnoreCase(EXIT_COMMAND));

        scanner.close();
        System.out.println("Program ended.");
    }

    // Process the user's command
    private static void processCommand(String command, List<Command> history) {
        String[] parts = command.split(" ");
        if (parts[0].equalsIgnoreCase(ADD_COMMAND) && parts.length > 1) {
            processAddCommand(parts[1], history);
        } else if (parts[0].equalsIgnoreCase(SUBTRACT_COMMAND) && parts.length > 1) {
            processSubtractCommand(parts[1], history);
```

```
        } else if (parts[0].equalsIgnoreCase(MULTIPLY_COMMAND) && parts.length > 1) {
            processMultiplyCommand(parts[1], history);
        } else if (parts[0].equalsIgnoreCase(DIVIDE_COMMAND) && parts.length > 1) {
            processDivideCommand(parts[1], history);
        } else if (parts[0].equalsIgnoreCase(RESULT_COMMAND)) {
            displayResult(calculateResult(history));
        } else if (parts[0].equalsIgnoreCase(CLEAR_COMMAND)) {
            history.clear();
            System.out.println("Result and history cleared.");
        } else if (parts[0].equalsIgnoreCase(HISTORY_COMMAND)) {
            displayHistory(history);
        } else if (parts[0].equalsIgnoreCase(REMOVE_COMMAND) && parts.length > 1) {
            processRemoveCommand(parts[1], history);
        } else {
            System.out.println("Unrecognized command. Please enter a valid command.");
        }
    }

    // Process the add command
    private static void processAddCommand(String numberString, List<Command> history) {
        try {
            double numberToAdd = Double.parseDouble(numberString);
            history.add(new Command(ADD_COMMAND, numberToAdd));
            System.out.println("New result: " + calculateResult(history));
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to add.");
        }
    }

    // Process the subtract command
    private static void processSubtractCommand(String numberString, List<Command>
history) {
        try {
            double numberToSubtract = Double.parseDouble(numberString);
            history.add(new Command(SUBTRACT_COMMAND, numberToSubtract));
            System.out.println("New result: " + calculateResult(history));
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to
subtract.");
        }
    }

    // Process the multiply command
    private static void processMultiplyCommand(String numberString, List<Command>
```

```
    private static void processMultiplyCommand(String numberString, List<Command>
history) {
        try {
            double numberToMultiply = Double.parseDouble(numberString);
            history.add(new Command(MULTIPLY_COMMAND, numberToMultiply));
            System.out.println("New result: " + calculateResult(history));
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to multiply.");
        }
    }

    // Process the divide command
    private static void processDivideCommand(String numberString, List<Command>
history) {
        try {
            double numberToDivide = Double.parseDouble(numberString);
            if (numberToDivide == 0) {
                System.out.println("Error: Division by zero is not allowed.");
            } else {
                history.add(new Command(DIVIDE_COMMAND, numberToDivide));
                System.out.println("New result: " + calculateResult(history));
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format. Please enter a valid number to divide.");
        }
    }

    // Process the remove command
    private static void processRemoveCommand(String indexString, List<Command>
history) {
        try {
            int indexToRemove = Integer.parseInt(indexString);
            if (indexToRemove < 0 || indexToRemove >= history.size()) {
                System.out.println("Error: Index out of bounds. Please enter a valid index to
remove.");
            } else {
                history.remove(indexToRemove);
                System.out.println("Command removed from history.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid index format. Please enter a valid index to remove.");
        }
    }

    // Calculate the result based on the history of commands
```

```java
    // Calculate the result based on the history of commands
    private static double calculateResult(List<Command> history) {
        double result = 0.0;
        for (Command command : history) {
            switch (command.getName()) {
                case ADD_COMMAND:
                    result += command.getNumber();
                    break;
                case SUBTRACT_COMMAND:
                    result -= command.getNumber();
                    break;
                case MULTIPLY_COMMAND:
                    result *= command.getNumber();
                    break;
                case DIVIDE_COMMAND:
                    if (command.getNumber() != 0) {
                        result /= command.getNumber();
                    }
                    break;
            }
        }
        return result;
    }

    // Display the current result
    private static void displayResult(double result) {
        System.out.println("Current result: " + result);
    }

    // Display the history of commands
    private static void displayHistory(List<Command> history) {
        if (history.isEmpty()) {
            System.out.println("No history available.");
        } else {
            System.out.println("History of commands:");
            for (int i = 0; i < history.size(); i++) {
                System.out.println(i + ": " + history.get(i));
            }
        }
    }
}
```

Final Code Solution to Part 2, Command.java

```java
package com.training4programmers;

public class Command {
    private String name;
    private double number;

    public Command(String name, double number) {
        this.name = name;
        this.number = number;
    }

    public String getName() {
        return name;
    }

    public double getNumber() {
        return number;
    }

    @Override
    public String toString() {
        return "Command: " + name + ", Number: " + number;
    }
}
```

Build a Calculator with C#

Setup

To complete the following workshop, please create a C# console application using .NET 8, with top-level statements enabled.

Part 1: Basic Calculator Steps

Note: For Part 1, sample Copilot chat prompts and suggested code are provided. For Part 2 and Part 3, you will need to create your own chat prompts and use the suggestions from Copilot to update the code.

1. Create the new project and open the project with Visual Studio (not Visual Studio Code).
2. Open Copilot Chat, and explain to Copilot that you want to build a console-based calculator in C#.

Ask Copilot to help you write the code. Start by asking Copilot to write a command loop that will keep asking the user for input until the user types "exit". For any command other than "exit", print the command in the console. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```text
I am working on new C# console-based Calculator program. I will need some help writing the code. Can you please create a basic command loop that will ask users for a new command until the user types "exit". When the user types "exit" the program should end. For any other comment, please display the command in the console before asking for the next command. I am using top-level statements in the Program.cs file.
```

Copilot Suggested Code

```C#
string command = "";
while (command != "exit")
{
    Console.Write("Enter a command: ");
    command = Console.ReadLine();
    if (command != "exit")
    {
        Console.WriteLine("You entered: " + command);
    }
}
```

3. Next, ask Copilot to track the current calculator state in a variable named "result". Ask Copilot to initialize the variable to 0. Then, ask Copilot to add a new command to add a number to the result. When the user types "add 5", for example, the result should be updated to 5. If the user types "add 3", the result should be updated to 8. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```text
Track current state of the calculator with a result variable initialized to 0. The result should be a floating point number. Also, please add one command for adding a number to the result. The command should display the new result after performing the add operation. The format of the command should be similar to this "add 3" which would then add 3 to the result and display the new result.
```

Copilot Suggested Code

```C#
double result = 0;
string command = "";
while (command != "exit")
{
    Console.Write("Enter a command: ");
    command = Console.ReadLine();
    if (command.StartsWith("add "))
    {
        double numberToAdd;
        if (double.TryParse(command.Substring(4), out numberToAdd))
        {
            result += numberToAdd;
            Console.WriteLine("New result: " + result);
        }
        else
        {
            Console.WriteLine("Invalid number format.");
        }
    }
    else if (command != "exit")
    {
        Console.WriteLine("You entered: " + command);
    }
}
```

4. Verify the code created so far works. Add a new command named "result" that will display the current result any time the users asks. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```TEXT
Add a command to display the current result. Also, if the user types an unrecognized command, please display an error message and allow the user to type another command.
```

Copilot Suggested Code

```C#
double result = 0;
string command = "";
while (command != "exit")
{
    Console.Write("Enter a command: ");
    command = Console.ReadLine();
    if (command.StartsWith("add "))
    {
        double numberToAdd;
        if (double.TryParse(command.Substring(4), out numberToAdd))
        {
            result += numberToAdd;
            Console.WriteLine("New result: " + result);
        }
        else
        {
            Console.WriteLine("Invalid number format.");
        }
    }
    else if (command == "result")
    {
        Console.WriteLine("Current result: " + result);
    }
    else if (command != "exit")
    {
        Console.WriteLine("Unrecognized command: " + command);
    }
}
```

Sample Copilot Chat Prompt

```TEXT
Please refactor the code to make it more readable. Perhaps, the code should be broken into functions or classes.
```

Copilot Suggested Code

C#

```
class Calculator
{
  private double result = 0;

  public void Run()
  {
    string command = "";
    while (command != "exit")
    {
      Console.Write("Enter a command: ");
      command = Console.ReadLine();
      ProcessCommand(command);
    }
  }

  private void ProcessCommand(string command)
  {
    if (command.StartsWith("add "))
    {
      AddToResult(command);
    }
    else if (command == "result")
    {
      DisplayResult();
    }
    else if (command != "exit")
    {
      Console.WriteLine("Unrecognized command: " + command);
    }
  }

  private void AddToResult(string command)
  {
    double numberToAdd;
    if (double.TryParse(command.Substring(4), out numberToAdd))
    {
      result += numberToAdd;
      Console.WriteLine("New result: " + result);
    }
    else
    {
      Console.WriteLine("Invalid number format.");
```

```
    }
  }

  private void DisplayResult()
  {
    Console.WriteLine("Current result: " + result);
  }
}

var calculator = new Calculator();
calculator.Run();
```

The suggested code has a bug. Do you know what the bug is? Because we are using top-level statements, the code instantiating and running the Calculator must be located at the top of the file. Feel free to move the code to the top, or discuss with Copilot how to fix the bug.

The fixed code looks like this:

Program.cs

```C#
var calculator = new Calculator();
calculator.Run();

class Calculator
/* reaming code */
```

6. Next, ask Copilot to add new commands for subtraction, multiplication, and division. Ensure there is the error handling for division by zero. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

```TEXT
Please add commands for subtraction, multiplication, and division. Also, ensure there is error handling for division by zero.
```

Copilot Suggested Code

C#

```
var calculator = new Calculator();
calculator.Run();

class Calculator
{
 private double result = 0;

 public void Run()
 {
  string command = "";
  while (command != "exit")
  {
   Console.Write("Enter a command: ");
   command = Console.ReadLine();
   ProcessCommand(command);
  }
 }

 private void ProcessCommand(string command)
 {
  if (command.StartsWith("add "))
  {
   PerformOperation(command, (a, b) => a + b);
  }
  else if (command.StartsWith("subtract "))
  {
   PerformOperation(command, (a, b) => a - b);
  }
  else if (command.StartsWith("multiply "))
  {
   PerformOperation(command, (a, b) => a * b);
  }
  else if (command.StartsWith("divide "))
  {
   PerformOperation(command, (a, b) => b != 0 ? a / b : throw new
DivideByZeroException());
  }
  else if (command == "result")
  {
   DisplayResult();
  }
  else if (command != "exit")
```

```
    {
      Console.WriteLine("Unrecognized command: " + command);
    }
  }

  private void PerformOperation(string command, Func<double, double, double> operation)
  {
    double number;
    if (double.TryParse(command.Substring(command.IndexOf(' ') + 1), out number))
    {
      try
      {
        result = operation(result, number);
        Console.WriteLine("New result: " + result);
      }
      catch (DivideByZeroException)
      {
        Console.WriteLine("Error: Division by zero.");
      }
    }
    else
    {
      Console.WriteLine("Invalid number format.");
    }
  }

  private void DisplayResult()
  {
    Console.WriteLine("Current result: " + result);
  }
}
```

If Copilot places the `Calculator` instantiation and `Run` method call at the bottom of the file, move it to the top of the `Program.cs` file.

7. Finally, ask Copilot to add a command for clearing the result. When the user types "clear", the result should be reset to 0. Using the suggestions from Copilot, update the code.

Sample Copilot Chat Prompt

TEXT

Please add a command for clearing the result. When the user types "clear", the result should be reset to 0.

Copilot Suggested Code

C#

```
var calculator = new Calculator();
calculator.Run();

class Calculator
{
 private double result = 0;

 public void Run()
 {
  string command = "";
  while (command != "exit")
  {
   Console.Write("Enter a command: ");
   command = Console.ReadLine();
   ProcessCommand(command);
  }
 }

 private void ProcessCommand(string command)
 {
  if (command.StartsWith("add "))
  {
   PerformOperation(command, (a, b) => a + b);
  }
  else if (command.StartsWith("subtract "))
  {
   PerformOperation(command, (a, b) => a - b);
  }
  else if (command.StartsWith("multiply "))
  {
   PerformOperation(command, (a, b) => a * b);
  }
  else if (command.StartsWith("divide "))
  {
   PerformOperation(command, (a, b) => b != 0 ? a / b : throw new
DivideByZeroException());
  }
  else if (command == "result")
  {
   DisplayResult();
  }
  else if (command == "clear")
```

```
42      {
43        ClearResult();
44      }
45      else if (command != "exit")
46      {
47        Console.WriteLine("Unrecognized command: " + command);
48      }
49    }
50
51    private void PerformOperation(string command, Func<double, double, double> operation)
52    {
53      double number;
54      if (double.TryParse(command.Substring(command.IndexOf(' ') + 1), out number))
55      {
56        try
57        {
58          result = operation(result, number);
59          Console.WriteLine("New result: " + result);
60        }
61        catch (DivideByZeroException)
62        {
63          Console.WriteLine("Error: Division by zero.");
64        }
65      }
66      else
67      {
68        Console.WriteLine("Invalid number format.");
69      }
70    }
71
72    private void DisplayResult()
73    {
74      Console.WriteLine("Current result: " + result);
75    }
76
77    private void ClearResult()
78    {
79      result = 0;
80      Console.WriteLine("Result cleared.");
81    }
82  }
```

If Copilot places the `Calculator` instantiation and `Run` method call at the bottom of the file, move it to

the top of the `Program.cs` file.

Part 2: Calculator History Steps

Note: For Part 2, the steps are listed, but no sample prompts or code are provided. You will need to create your own chat prompts and use the suggestions from Copilot to update the code.

1. Ask Copilot to track the history of all commands entered by the user. When the user types "history", the console should display all the commands entered so far. Using the suggestions from Copilot, update the code.
2. Add a command to remove a history entry from the list of commands. When the user types "remove 3", for example, the third command should be removed from the history. Using the suggestions from Copilot, update the code.
3. Ask Copilot how to modify the code to display the result based upon the `history` list not the `result` variable. Remove the `result` variable from the code. Using the suggestions from Copilot, update the code.

Note: The solution to part 2 is located at the of this workshop.

Part 3: History File Operations Steps

Note: For Part 3, a user story describes your task. Complete the user story using your AI pair-programming skills. Unlike Part 1 and 2, there is no solution provided for Part 3. Good luck!

User Story: Persist and Reload Calculator Command History

As a user of the calculator application, I want the ability to persist the history of my calculator commands to a file and reload those commands from the file back into the calculator. This will allow me to save my calculation history for future reference and easily restore my previous calculations when needed.

Acceptance Criteria:
1. The calculator application should provide an option to save the command history to a file.
  - The user should be able to specify the location and name of the file to save the command history.
  - The command history should be saved in JSON.
  - The user should receive confirmation that the command history has been successfully saved to the file.

2. The calculator application should provide an option to load a previously saved command history file.
  - The user should be able to select the file containing the command history to be loaded.
  - The application should validate the file format and ensure that it contains valid command history data.
  - If the file is valid, the command history should be loaded into the calculator, replacing any existing command history.
  - The user should receive confirmation that the command history has been successfully loaded from the file.

3. The loaded command history should be accessible and usable within the calculator.
  - The user should be able to view the loaded command history, including all previous commands and their results.
  - The user should be able to navigate through the loaded command history, similar to the existing functionality for navigating the command history within the calculator.
  - The user should be able to execute commands from the loaded command history, as if they were entered manually.

4. Do not worry about implementing error handling for reading and writing to the file.

Solution for Part 2

Copilot Chat Prompt #1

TEXT

Enhance the application to remember only the history of math commands (add, subtract, multiply, divide) typed by the user. Each command in the history should remember the command name and the number entered by the user. Also, include a new command named "history" that will display the history of math operations. The clear command should clear the history.

Copilot Chat Prompt #2

```
TEXT
Enhance the code to remove an entry from the history using a new command named "remove". The structure for the command is "remove 3" where 3 is the index of the command to remove. If there is no command at that index, please display an error message and allow the user to enter another command.
```

Copilot Chat Prompt #3

```
TEXT
Remove the result variable, and calculate the result using the history of math commands.
```

Final Code Solution to Part 2

C#

```
var calculator = new Calculator();
calculator.Run();

class Calculator
{
  private List<(string operation, double number)> commandHistory = new List<(string
operation, double number)>();

  public void Run()
  {
    string command = "";
    while (command != "exit")
    {
      Console.Write("Enter a command: ");
      command = Console.ReadLine();
      ProcessCommand(command);
    }
  }

  private void ProcessCommand(string command)
  {
    if (command.StartsWith("add "))
    {
      PerformOperation(command, (a, b) => a + b);
    }
    else if (command.StartsWith("subtract "))
    {
      PerformOperation(command, (a, b) => a - b);
    }
    else if (command.StartsWith("multiply "))
    {
      PerformOperation(command, (a, b) => a * b);
    }
    else if (command.StartsWith("divide "))
    {
      PerformOperation(command, (a, b) => b != 0 ? a / b : throw new
DivideByZeroException());
    }
    else if (command == "result")
    {
      DisplayResult();
    }
```

```
42      else if (command == "clear")
43      {
44        ClearResult();
45      }
46      else if (command == "history")
47      {
48        DisplayHistory();
49      }
50      else if (command.StartsWith("remove "))
51      {
52        RemoveCommand(command);
53      }
54      else if (command != "exit")
55      {
56        Console.WriteLine("Unrecognized command: " + command);
57      }
58    }
59
60    private void PerformOperation(string command, Func<double, double, double> operation)
61    {
62      double number;
63      if (double.TryParse(command.Substring(command.IndexOf(' ') + 1), out number))
64      {
65        try
66        {
67          commandHistory.Add((command.Substring(0, command.IndexOf(' ')), number));
68          Console.WriteLine("New command added: " + command);
69        }
70        catch (DivideByZeroException)
71        {
72          Console.WriteLine("Error: Division by zero.");
73        }
74      }
75      else
76      {
77        Console.WriteLine("Invalid number format.");
78      }
79    }
80
81    private void DisplayResult()
82    {
83      double result = 0;
84      foreach (var command in commandHistory)
85      {
```

```
    {
      switch (command.operation)
      {
        case "add":
          result += command.number;
          break;
        case "subtract":
          result -= command.number;
          break;
        case "multiply":
          result *= command.number;
          break;
        case "divide":
          if (command.number != 0)
          {
            result /= command.number;
          }
          else
          {
            Console.WriteLine("Error: Division by zero in history.");
            return;
          }
          break;
      }
    }
    Console.WriteLine("Current result: " + result);
  }

  private void ClearResult()
  {
    commandHistory.Clear();
    Console.WriteLine("History cleared.");
  }

  private void DisplayHistory()
  {
    Console.WriteLine("Command history:");
    foreach (var command in commandHistory)
    {
      Console.WriteLine($"{command.operation} {command.number}");
    }
  }

  private void RemoveCommand(string command)
```

```
    {
      int index;
      if (int.TryParse(command.Substring(7), out index))
      {
        if (index >= 0 && index < commandHistory.Count)
        {
          commandHistory.RemoveAt(index);
          Console.WriteLine("Command removed from history.");
        }
        else
        {
          Console.WriteLine("Error: No command at that index.");
        }
      }
      else
      {
        Console.WriteLine("Invalid index format.");
      }
    }
  }
```

Conclusion

Thank you for attending the course! If you have any questions, please get in touch with Eric Greene (https://www.linkedin.com/in/ericwgreene). We hope you enjoyed the course and learned a lot about GitHub Copilot.

We look forward to serving you in a future course!

Last updated 2024-03-16 17:54:49 UTC